كتاب ملوّن خطوة بخطوة للمبتدئين

برمجة بايثون

كأن لم تعرفها من قبل

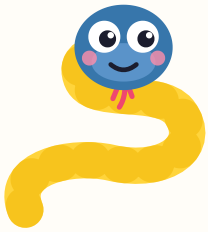
من أوّل سطر برمجي إلى أوّل مشروع حقيقي، بلغة بسيطة وأمثلة ملوّنة



من الصفر | تمارين وتحدّيات | أمثلة من الصناعة | مشروع حقيقي

تأليف د. سفيان مرابطي

Dr Merabti



برمجة بايثون

كأن لم تعرفها من قبل

دليلك المصوّر خطوة بخطوة، من الصفر إلى أوّل مشروع حقيقي

تأليف

د. سفيان مرابطي

برمجة بايثون كأن لم تعرفها من قبل — د. سفيان مرابطي

الطبعة الأولى، 2026

جميع الحقوق محفوظة للمؤلف. لا يجوز نسخ هذا الكتاب أو توزيعه أو بيعه، كليًّا أو جزئيًّا، بأي وسيلة دون إذن كتابي من المؤلف.

Python اسم وعلامة مسجّلة لمؤسسة Python Software Foundation، ولا علاقة لها بهذا الكتاب.

merabti.com

مرحبًا بك يا صديقي

هل سمعت كلمة «برمجة» فتخيّلت شاشة سوداء مليئة بالرموز الغامضة، وشخصًا عبقريًّا لا ينام؟ انسَ هذه الصورة تمامًا. البرمجة ببساطة هي أن **تتحدّث مع الحاسوب بلغة يفهمها**، فتطلب منه أن يحسب أو يرتّب أو يرسم أو يكرّر عملًا مملًّا بدلًا منك. ولغة بايثون هي أسهل هذه اللغات وأقربها إلى لغة البشر.

هذا الكتاب كُتب لمن **لم يكتب سطرًا برمجيًّا واحدًا في حياته**: الطالب والمهندس والتقني والأستاذ وصاحب المشروع، بل وحتى من جرّب البرمجة من قبل فتعقّدت أمامه، فأراد أن يبدأ من جديد «كأن لم يعرفها من قبل».

1 كيف صُمّم هذا الكتاب؟

الفهرس

كل فصل يبدأ بصفحة ملوّنة تخبرك بما ستتعلّمه، ثم يأتي الشرح خطوة خطوة، ومع كل فكرة مثال برمجي تستطيع كتابته وتجربته فورًا، ثم ترى **الناتج** الذي سيظهر على شاشتك. وفي نهاية كل فصل خلاصة سريعة وتحدٍّ صغير تجد حلّه في آخر الكتاب.

الأمثلة البرمجية تظهر هكذا، في صندوق داكن مثل محرّر الأكواد الحقيقي، مع أرقام الأسطر:

Python `python`

```python
name = "سفيان"
print("أهلًا", name)
print("مرحبًا بك في عالم بايثون!")
```

أما ما يظهر على الشاشة بعد تشغيل البرنامج فيكون في صندوق أخضر:

الناتج

```
أهلًا سفيان
مرحبًا بك في عالم بايثون!
```

2 رموز ستلازمك في الطريق

الفهرس

نصيحة ذهبية

حيلة أو عادة جيدة توفّر عليك الوقت وتجعل برامجك أجمل.

انتبه!

خطأ شائع يقع فيه المبتدئون، فانتبه له قبل أن تقع فيه.

معلومة

معلومة إضافية لطيفة تفهم بها ما يجري «خلف الكواليس».

جرّب بنفسك

تمرين قصير: أغلق الكتاب وجرّبه بنفسك على الحاسوب قبل أن تكمل.

من أرض المصنع

أمثلة من عالم الصناعة والطاقة: محرّكات، حسّاسات، استهلاك كهرباء... لترى أن بايثون أداة عمل حقيقية.

تحدّي الفصل

تحدٍّ أكبر في آخر كل فصل، وحلّه في ملحق الحلول.

3 تعرّف على «بايثو»

الفهرس

سيرافقك في هذه الرحلة صديق صغير اسمه **بايثو**، ثعبان لطيف لا يعضّ أبدًا (إلا الأخطاء البرمجية!). يظهر بين الحين والآخر ليعلّق أو ينصح أو يمزح قليلًا.

بايثو يقول:

مرحبًا! أنا بايثو. سمعت أنك تخاف من البرمجة؟ لا تقلق، أنا أيضًا كنت أخاف من الفئران... ثم أصبحت ثعبانًا. كل شيء ممكن!



4 نصيحتي الوحيدة قبل أن نبدأ

الفهرس

لا تقرأ هذا الكتاب كما تقرأ رواية. البرمجة مثل السباحة وقيادة الدراجة: لا تُتعلَّم بالقراءة وحدها. افتح الحاسوب بجانب الكتاب، واكتب كل مثال بيدك (لا تنسخه!)، وغيّر فيه، وأخطئ عمدًا لترى ماذا يحدث. فالخطأ هنا لا يكسر شيئًا، بل هو أفضل معلّم.

اضغط على أي عنوان في الفهرس لتنتقل إلى صفحته، واضغط على رأس أي صفحة (الشريط العلوي) أو رقمها في الأسفل لتعود إلى الفهرس.

أتمنّى لك رحلة ممتعة، وإلى اللقاء في السطر الأول!

الفهرس

اضغط على أي عنوان لتنتقل إلى صفحته، واضغط على رأس أي صفحة لتعود إلى هنا.

مرحبًا بك يا صديقي 3

الفصل 1
1 مرحبًا بايثون 10

- ما هي البرمجة أصلًا؟ 11
- لماذا بايثون بالذات؟ 11
- تثبيت بايثون على حاسوبك 12
- أين نكتب البرامج؟ 12
- أوّل برنامج لك 13
- البرامج في ملفات 13
- التعليقات: ملاحظات لك لا للحاسوب 14
- عندما يحدث خطأ 14

الفصل 2
2 المتغيّرات وأنواع البيانات 16

- ما هو المتغيّر؟ 17
- المتغيّر يتغيّر! 17
- كيف نسمّي المتغيّرات؟ 18
- أنواع البيانات الأساسية 18
- الدالة type: ما نوع هذه القيمة؟ 19
- التحويل بين الأنواع 20

الفصل 3
3 العمليات والحساب 22

- العمليات الحسابية 23
- ترتيب العمليات 24
- الاختصارات الذكية 24
- المقارنة: صح أم خطأ؟ 25
- العمليات المنطقية 25
- مكتبة math 26

الفصل 4
4 النصوص 28

- إنشاء النصوص 29
- اللصق والتكرار 29
- سلاسل f: أجمل طريقة لتركيب الجمل 30
- الفهرسة: كل حرف له رقم 30
- القصّ: أخذ جزء من النص 31
- دوال النصوص المفيدة 32

الفصل 5
5 الحوار مع المستخدم 34

- الدالة input 35
- انتبه: input تعيد نصًّا دائمًا! 35
- أسرار الدالة print 36
- تنسيق الأرقام والجداول 37
- مشروع صغير: حاسبة قانون أوم 37

الفصل 6
6 الشروط واتخاذ القرار 40

- الجملة if 41
- الإزاحة: قانون بايثون الذهبي 41
- else: وإلّا... 42
- elif: عندما تكثر الخيارات 42
- الشروط المركّبة 43
- الشرط في سطر واحد 44
- الجملة match 44

الفصل 7
7 الحلقات والتكرار 46

- لماذا نحتاج الحلقات؟ 47
- الحلقة for مع range 47
- المرور على النصوص 48
- الحلقة while 49
- break و continue 49
- الحلقات المتداخلة 50
- مشروع: لعبة «خمّن الرقم» 50

الفصل 8
8 القوائم والصفوف 53

- ما هي القائمة؟ 54
- الوصول إلى العناصر 54
- الإضافة والحذف 55
- الترتيب والبحث والإحصاء 56
- المرور على القوائم 56
- فهم القوائم: سطر واحد يغني عن أربعة 57
- الصفوف tuple 58

الفصل 9
9 القواميس والمجموعات 60

- ما هو القاموس؟ 61
- القراءة بأمان باستعمال get 61
- الإضافة والتعديل والحذف 62
- المرور على القواميس 62
- القواميس المتداخلة 63
- المجموعات set 64

الفصل 10
10 الدوال 66

- لماذا الدوال؟ 67
- تعريف الدالة واستدعاؤها 67
- المعاملات: نعطي الدالة معلومات 68
- return: الدالة تعيد نتيجة 68
- القيم الافتراضية والمعاملات المسمّاة 69
- المجال: أين يعيش المتغيّر؟ 69
- التوثيق و lambda 70

الفصل 11
11 الوحدات والمكتبات 72

- ما هي الوحدة؟ 73
- طرق الاستيراد 73
- جولة في المكتبات المدمجة 74
- صنع وحدتك الخاصة 75
- تثبيت المكتبات الخارجية بـ pip 76
- لمحة: ارسم أوّل منحنى لك 77

الفصل 12
12 الملفات ومعالجة الأخطاء 78

- لماذا الملفات؟ 79
- الكتابة في ملف 79
- القراءة من ملف 80
- ملفات CSV 80
- الأخطاء الشائعة 81
- try و except: شبكة الأمان 81
- إدخال لا يتعطّل أبدًا 82

الفصل 13
13 البرمجة الكائنية 85

- الفكرة: الصنف والكائن 86
- أوّل صنف لك 86
- التوابع: ماذا يستطيع الكائن أن يفعل؟ 87
- الطباعة الجميلة بـ __str__ 89
- الوراثة 89

الفصل 14
14 المشروع النهائي: مراقب الطاقة 92

- فكرة المشروع 93
- الخطوة 1: ملف البيانات 93
- الخطوة 2: الصنف Motor 94
- الخطوة 3: قراءة الملف بأمان 94
- الخطوة 4: التقرير وفرص التوفير 95
- الخطوة 5: الحفظ والقائمة الرئيسية 96
- لحظة الحقيقة: التشغيل 97

الورقة المرجعية السريعة 99

حلول التحدّيات 103

ماذا بعد؟ 112

الفصل

01

مرحبًا بايثون

ما البرمجة؟ ولماذا بايثون؟ وكيف تكتب أوّل برنامج لك في أقل من خمس دقائق

في هذا الفصل ستتعلّم

- ما معنى البرمجة ولماذا نحتاجها
- تثبيت بايثون على حاسوبك خطوة خطوة
- الفرق بين الوضع التفاعلي والملفات
- لماذا اختار العالم لغة بايثون
- كتابة أوّل برنامج وتشغيله
- كتابة التعليقات داخل البرنامج



كل مبرمج عظيم بدأ بسطر واحد: print("Hello"). اليوم دورك!

1.1 ما هي البرمجة أصلًا؟

الفهرس

الحاسوب آلة سريعة جدًّا لكنها **لا تفكّر**. إنها تنفّذ الأوامر حرفيًّا، واحدًا بعد الآخر، بلا ملل ولا تعب. البرنامج هو مجموعة من هذه الأوامر مكتوبة بترتيب معيّن، مثل وصفة طبخ:

1. اغسل الطماطم.
2. قطّعها إلى مكعّبات.
3. أضف الملح.
4. إذا كانت حامضة، أضف قليلًا من السكر.

لاحظ أن الوصفة فيها **خطوات متتالية** و**شرط** («إذا كانت حامضة»). البرامج كلها مبنيّة من هذه الأفكار البسيطة نفسها، وستتعلّمها كلها في هذا الكتاب.

1.2 لماذا بايثون بالذات؟

الفهرس

ظهرت لغة بايثون سنة 1991 على يد المبرمج الهولندي **غيدو فان روسم** (Guido van Rossum)، وأخذت اسمها من برنامج كوميدي بريطاني اسمه Monty Python، لا من الثعبان! واليوم هي من أكثر لغات البرمجة استعمالًا في العالم، وذلك لأسباب كثيرة:

- **سهلة القراءة**: تكاد تُقرأ كأنها جمل إنجليزية بسيطة.
- **مجانية** ومفتوحة المصدر، وتعمل على ويندوز وماك ولينكس.
- **متعدّدة الاستعمالات**: الذكاء الاصطناعي، تحليل البيانات، المواقع، الأتمتة، العلوم والهندسة، التحكّم في الأجهزة...
- **مجتمع ضخم**: أيّ سؤال يخطر ببالك، ستجد أن آلاف الناس سألوه قبلك وأجيبوا عنه.

قارن بين برنامج يطبع جملة على الشاشة بلغة جافا وبلغة بايثون:

اللغة	البرنامج
جافا	`public class A { public static void main(String[] a) { System.out.println("Hi"); } }`
بايثون	`print("Hi")`

سطر واحد فقط! هذا هو سحر بايثون.

من أرض المصنع

في المصانع الحديثة تُستعمل بايثون لجمع قراءات الحسّاسات، ورسم منحنيات استهلاك الطاقة، وكشف الأعطال قبل وقوعها بالذكاء الاصطناعي (الصيانة التنبّؤية)، وكتابة التقارير اليومية آليًّا بدل نسخها يدويًّا في إكسل.

1.3 تثبيت بايثون على حاسوبك

الفهرس

على ويندوز

1. ادخل إلى الموقع الرسمي `python.org` واضغط على **Downloads**.
2. حمّل آخر إصدار (يبدأ بالرقم 3، مثل Python 3.13).
3. شغّل الملف الذي حمّلته.
4. **مهم جدًّا:** فعّل الخيار **Add python.exe to PATH** في أسفل النافذة الأولى.
5. اضغط **Install Now** وانتظر حتى ينتهي التثبيت.

انتبه!

إذا نسيت تفعيل خيار **Add to PATH** فلن يتعرّف الحاسوب على الأمر `python` في نافذة الأوامر. الحل: أعد تشغيل ملف التثبيت واختر **Modify** ثم فعّل الخيار.

هل نجح التثبيت؟

افتح نافذة الأوامر: اضغط `Win` + `R` ثم اكتب `cmd` واضغط `Enter`، ثم اكتب:

```
Terminal
python --version
```

الناتج

```
Python 3.13.0
```

إذا ظهر رقم الإصدار فمبروك، بايثون موجودة على حاسوبك! (على ماك ولينكس اكتب `python3` بدل `python`).

1.4 أين نكتب البرامج؟

الفهرس

مع بايثون يأتي برنامج صغير اسمه **IDLE**، وهو كافٍ تمامًا للبداية. ابحث عنه في قائمة ابدأ. وعندما تتقدّم قليلًا أنصحك بمحرّر **Visual Studio Code** المجاني، فهو يلوّن الأكواد ويكتشف الأخطاء ويكمل الكلمات بدلًا منك.

> **معلومة**
>
> يمكنك أيضًا البرمجة دون تثبيت أيّ شيء عبر مواقع مثل **Google Colab** أو `replit.com` ، تكتب فيها الكود في المتصفّح وتشغّله مباشرة. مفيد جدًّا إذا كنت تتعلّم من الهاتف أو من حاسوب العمل.

1.5 أوّل برنامج لك

الفهرس

افتح IDLE، ستظهر نافذة فيها ثلاث علامات `>>>` هذا هو **الوضع التفاعلي**: تكتب أمرًا فتنفّذه بايثون فورًا. اكتب:

Python Shell — python

```
>>> print("مرحبًا يا عالم!")
مرحبًا يا عالم!
```

ها قد كتبت أول برنامج لك! الأمر `print` معناه «اطبع»، وكل ما تضعه بين القوسين وعلامتَي التنصيص يظهر على الشاشة كما هو.

جرّب الآن أن تستعمل بايثون كآلة حاسبة:

Python Shell — python

```
>>> 7 + 5
12
>>> 230 * 15
3450
>>> 100 / 3
33.333333333333336
```



بايثو يقول:

لاحظ أن بايثون لا تتعب من الحساب أبدًا. اطلب منها `2 ** 1000` وانظر إلى الرقم العملاق! (العلامة `**` تعني «أُس»).

1.6 البرامج في ملفات

الفهرس

الوضع التفاعلي رائع للتجربة، لكن ما تكتبه فيه يضيع عند إغلاق النافذة. البرامج الحقيقية تُكتب في **ملفات** تنتهي بـ `.py` .

1. في IDLE اختر **File** ← **New File**.

2. اكتب البرنامج التالي.
3. احفظه باسم `hello.py` عبر `Ctrl` `S`.
4. شغّله بالضغط على `F5`.

hello.py — python

```python
print("السلام عليكم")
print("اسمي بايثون، وعمري أكثر من 30 سنة")
print("وأنا هنا لأساعدك!")
```

الناتج

```
السلام عليكم
اسمي بايثون، وعمري أكثر من 30 سنة
وأنا هنا لأساعدك!
```

لاحظ أن بايثون تنفّذ الأسطر **بالترتيب من الأعلى إلى الأسفل**، سطرًا بعد سطر.

1.7 التعليقات: ملاحظات لك لا للحاسوب

الفهرس

كل ما يأتي بعد العلامة `#` يتجاهله الحاسوب تمامًا. نستعمل ذلك لكتابة **تعليقات** تشرح الكود لنا ولغيرنا:

Python — python

```python
# هذا البرنامج يحسب مساحة غرفة
print(4 * 5)  # الطول × العرض
# print("هذا السطر لن يُنفَّذ")
```

الناتج

```
20
```

> **نصيحة ذهبية**
>
> اكتب التعليقات لتشرح **لماذا** كتبت الكود، لا **ماذا** يفعل. وصدّقني، بعد ستة أشهر ستنسى كل شيء، وستشكر نفسك على كل تعليق كتبته!

1.8 عندما يحدث خطأ

الفهرس

جرّب أن تكتب `Print` بحرف كبير، أو أن تنسى قوسًا:

Python Shell `python`

```
>>> Print("مرحبًا")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Print' is not defined
```

لا تخف من هذه الرسالة الحمراء! إنها ليست عقابًا، بل **مساعدة**: بايثون تخبرك بالسطر الذي فيه الخطأ (`line 1`) وبنوعه (`NameError` : اسم غير معروف). بايثون **تفرّق بين الحروف الكبيرة والصغيرة**، فـ `print` غير `Print` .

خلاصة الفصل

- البرنامج مجموعة أوامر يُنفّذها الحاسوب بالترتيب.
- `print()` تطبع على الشاشة، وبايثون تحسب مثل الآلة الحاسبة.
- الوضع التفاعلي `>>>` للتجربة، والملفات `.py` للبرامج الحقيقية.
- `#` لكتابة التعليقات، والحروف الكبيرة والصغيرة مختلفة.
- رسائل الخطأ صديقتك: اقرأ آخر سطر منها دائمًا.

تحدّي الفصل

اكتب برنامجًا في ملف اسمه `card.py` يطبع بطاقة تعريف لك من خمسة أسطر: اسمك، مهنتك، مدينتك، هوايتك، وسطرًا أخيرًا من النجوم `**********` .

الفصل

02

المتغيّرات وأنواع البيانات

صناديق صغيرة نحفظ فيها المعلومات لنستعملها متى شئنا

في هذا الفصل ستتعلّم

- ما هو المتغيّر وكيف نُنشئه
- الأعداد الصحيحة والعشرية
- معرفة نوع أيّ قيمة بالدالة type
- قواعد تسمية المتغيّرات
- النصوص والقيم المنطقية
- التحويل بين الأنواع



المتغيّر مثل علبة عليها اسم. ضع فيها ما تشاء، ثم نادِها باسمها!

2.1 ما هو المتغيّر؟

الفهرس

تخيّل أن أمامك علبًا صغيرة، تكتب على كل علبة اسمًا، وتضع داخلها شيئًا ما. هذا بالضبط هو **المتغيّر**: اسم نربطه بقيمة لنستعملها لاحقًا.

Python
python

```python
age = 25
name = "أمينة"
print(name)
print(age)
```

الناتج

```
أمينة
25
```

العلامة `=` في البرمجة **لا تعني «يساوي»** كما في الرياضيات، بل تعني «**ضع** القيمة التي على اليمين في الصندوق الذي على اليسار». لذلك نقرأ `age = 25` هكذا: «ضع 25 في المتغيّر age».

2.2 المتغيّر يتغيّر!

الفهرس

سُمّي متغيّرًا لأن قيمته يمكن أن تتغيّر في أي لحظة:

Python
python

```python
score = 10
print(score)
score = 15
print(score)
score = score + 5
print(score)
```

الناتج

```
10
15
20
```

السطر الخامس قد يبدو غريبًا: `score = score + 5`. بايثون تحسب الجهة اليمنى أولًا (15 + 5 = 20) ثم تضع النتيجة في `score`. القيمة القديمة تضيع وتحلّ محلها الجديدة.

معلومة

يمكنك إعطاء عدّة متغيّرات قيمًا في سطر واحد: `x, y, z = 1, 2, 3` وكذلك تبديل قيمتين بسهولة مذهلة: `x, y = y, x` . في أغلب اللغات الأخرى تحتاج متغيّرًا ثالثًا لتفعل ذلك!

2.3 كيف نسمّي المتغيّرات؟

الفهرس

للتسمية قواعد لا بدّ منها، وعادات حسنة يُستحسن اتباعها:

القاعدة	صحيح	خطأ
يبدأ بحرف أو `_`	`name` و `_count`	`2name`
بلا مسافات	`motor_speed`	`motor speed`
حروف وأرقام و `_` فقط	`temp2`	`price$` و `temp-2`
ليس كلمة محجوزة	`my_class`	`class` و `if` و `for`

نصيحة ذهبية

اختر أسماء **تشرح نفسها**: `temperature` أفضل بكثير من `t` ، و `total_price` أفضل من `tp` . والعادة في بايثون أن نكتب الأسماء بحروف صغيرة ونفصل الكلمات بشرطة سفلية، مثل `max_power` ، ويسمّى هذا الأسلوب snake_case، أي «أسلوب الثعبان»... طبعًا!

انتبه!

بايثون تسمح بالأسماء العربية مثل `العمر = 30` ، لكن لا أنصحك بذلك أبدًا، فهي تُربك المحرّرات وتختلط فيها الاتجاهات. اكتب الأسماء بالإنجليزية، والنصوص والتعليقات بالعربية كما تشاء.

2.4 أنواع البيانات الأساسية

الفهرس

ليست كل القيم من نوع واحد. فالعدد 25 ليس مثل الكلمة "مرحبًا". هذه الأنواع الأربعة الأساسية:

النوع	الاسم في بايثون	أمثلة
عدد صحيح	`int`	`7` و `-3` و `2026`
عدد عشري	`float`	`3.14` و `0.5` و `-12.75`
نص	`str`	`"سلام"` و `'Python'` و `"123"`
منطقي	`bool`	`True` و `False` فقط

Python
python

```python
voltage = 380          # int
current = 12.5         # float
motor = "مضخة الماء"   # str
running = True         # bool
```

انتبه!

العدد العشري يُكتب **بنقطة** لا بفاصلة: `12.5` صحيحة، أما `12,5` فبايثون تفهمها شيئًا آخر تمامًا (صفًّا من عددين، ستتعرّف عليه في الفصل الثامن).

2.5 الدالة type: ما نوع هذه القيمة؟

الفهرس

إذا شككت في نوع قيمة ما، اسأل بايثون مباشرة:

Python Shell
python

```
>>> type(25)
<class 'int'>
>>> type(25.0)
<class 'float'>
>>> type("25")
<class 'str'>
>>> type(True)
<class 'bool'>
```

لاحظ الفرق المهم: `25` عدد، أما `"25"` بين علامتَي تنصيص فهي **نص** مكوّن من حرفين، حتى لو كانا رقمين!

2.6 التحويل بين الأنواع

الفهرس

كثيرًا ما نحتاج إلى تحويل قيمة من نوع إلى آخر، وذلك باستعمال اسم النوع كدالة:

Python

```python
a = "50"
b = int(a)        # من نص إلى عدد صحيح
c = float(a)      # من نص إلى عدد عشري
d = str(3.7)      # من عدد إلى نص
print(b + 10)
print(c)
print(d + " كيلوواط")
```

الناتج

```
60
50.0
3.7 كيلوواط
```

انتبه!

لا يمكن تحويل أي نص إلى عدد: `int("مرحبًا")` يعطي خطأ `ValueError`، وكذلك `int("3.5")`. لتحويل `"3.5"` استعمل `float` أولًا.

من أرض المصنع

لوحات التحكّم وأجهزة القياس ترسل القراءات غالبًا **كنصوص**، مثل `"231.4"`. قبل أن تحسب بها المتوسّط أو تقارنها بحدّ الإنذار، يجب أن تحوّلها إلى أعداد باستعمال `float()`.



بايثو يقول:

سؤال سريع: ما نوع `True`؟ وما نوع `"True"`؟ إذا أجبت «bool ثم str» فأنت جاهز للفصل القادم!

خلاصة الفصل

- المتغيّر اسم يُشير إلى قيمة، ونعطيه القيمة بالعلامة `=` .
- الأسماء بالإنجليزية، بحروف صغيرة، وتشرح نفسها: `motor_speed` .
- الأنواع الأساسية: `int` و `float` و `str` و `bool` .
- `type()` تخبرك بالنوع، و `int()` و `float()` و `str()` تحوّل بين الأنواع.

تحدّي الفصل

أنشئ متغيّرات تصف محرّكًا كهربائيًّا: اسمه، قدرته بالكيلوواط (عدد عشري)، عدد أقطابه (عدد صحيح)، وهل هو يعمل الآن أم لا. ثم اطبع كل متغيّر مع نوعه باستعمال `type()` .

الفصل



03

العمليات والحساب

بايثون آلة حاسبة خارقة، وأكثر من ذلك بكثير

في هذا الفصل ستتعلّم

- العمليات الحسابية السبع
- الاختصارات مثل =+ و =-
- العمليات المنطقية and و or و not
- ترتيب العمليات والأقواس
- عمليات المقارنة
- مكتبة math للعمليات المتقدمة



هل تعلم أن 7 // 2 تساوي 3؟ لا تقلق، سأشرح لك السبب بعد قليل.

3.1 العمليات الحسابية

الفهرس

العملية	الرمز	المثال	النتيجة
الجمع	`+`	`7 + 2`	`9`
الطرح	`-`	`7 - 2`	`5`
الضرب	`*`	`7 * 2`	`14`
القسمة	`/`	`7 / 2`	`3.5`
القسمة الصحيحة	`//`	`7 // 2`	`3`
باقي القسمة	`%`	`7 % 2`	`1`
الأُس (القوة)	`**`	`7 ** 2`	`49`

العمليات الثلاث الأخيرة تحتاج إلى شرح قصير:

- **القسمة الصحيحة** `//` تعطي عدد المرّات الكاملة: في 7 دنانير كم قطعة من فئة 2 دينار؟ 3 قطع.
- **الباقي** `%` يعطي ما تبقّى بعد القسمة: يبقى دينار واحد.
- **الأُس** `**` : `2 ** 3` تعني 2 × 2 × 2 = 8.

Python

```python
minutes = 135
hours = minutes // 60
rest = minutes % 60
print(hours, "ساعة و", rest, "دقيقة")
```

الناتج

```
2 ساعة و 15 دقيقة
```

نصيحة ذهبية

الباقي `%` مفيد جدًّا لمعرفة إن كان العدد زوجيًّا: إذا كان `n % 2` يساوي `0` فالعدد زوجي. ستستعمل هذه الحيلة كثيرًا.

3.2 ترتيب العمليات

الفهرس

تتبع بايثون الترتيب نفسه الذي تعلّمته في المدرسة: الأقواس أولًا، ثم الأُس، ثم الضرب والقسمة، ثم الجمع والطرح.

Python Shell
python

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> 2 ** 3 ** 2
512
```

نصيحة ذهبية

عندما تشكّ في الترتيب، **ضع أقواسًا**. الأقواس لا تكلّف شيئًا، وتجعل الكود أوضح لك ولمن يقرأه بعدك.

3.3 الاختصارات الذكية

الفهرس

تغيير قيمة متغيّر بناءً على قيمته الحالية أمر شائع جدًّا، لذلك توجد له اختصارات:

الطويل	المختصر
`x = x + 5`	`x += 5`
`x = x - 2`	`x -= 2`
`x = x * 3`	`x *= 3`
`x = x / 2`	`x /= 2`

Python
python

```
1  counter = 0
2  counter += 1
3  counter += 1
4  print(counter)
```

الناتج

```
2
```

3.4 المقارنة: صح أم خطأ؟

الفهرس

عمليات المقارنة لا تعطي عددًا، بل تعطي قيمة منطقية: `True` (صح) أو `False` (خطأ).

الرمز	المعنى	المثال	النتيجة
`==`	يساوي	`5 == 5`	`True`
`≠`	لا يساوي	`5 ≠ 3`	`True`
`>`	أكبر من	`5 > 8`	`False`
`<`	أصغر من	`5 < 8`	`True`
`≥`	أكبر من أو يساوي	`5 ≥ 5`	`True`
`≤`	أصغر من أو يساوي	`9 ≤ 4`	`False`

انتبه!

أشهر خطأ عند المبتدئين: الخلط بين `=` و `==`. العلامة الواحدة `=` **تضع** قيمة في متغيّر، والعلامتان `==` **تسألان**: هل القيمتان متساويتان؟

3.5 العمليات المنطقية

الفهرس

نربط بها بين عدّة شروط:

- `and` (و): النتيجة صحيحة إذا كان **الشرطان معًا** صحيحين.
- `or` (أو): النتيجة صحيحة إذا كان **أحدهما على الأقل** صحيحًا.
- `not` (ليس): تقلب الصحيح خطأً والخطأ صحيحًا.

Python `python`

```python
temperature = 85
pressure = 4.2
print(temperature > 80 and pressure > 5)
print(temperature > 80 or pressure > 5)
print(not temperature > 80)
```

الناتج

```
False
True
False
```

معلومة

بايثون تسمح بكتابة المقارنات المتسلسلة كما في الرياضيات تمامًا: `18 ⩽ age ⩽ 60` ، وهذا أجمل من `age ⩾ 18 and age ⩽ 60` .

3.6 مكتبة math

الفهرس

لعمليات أكثر تقدّمًا، تأتي بايثون بمكتبة جاهزة اسمها `math` . نستدعيها بالكلمة `import` (وسنتحدّث عن المكتبات بالتفصيل في الفصل الحادي عشر):

Python

```python
import math

print(math.sqrt(144))        # الجذر التربيعي
print(math.pi)               # العدد π
print(round(math.pi, 2))     # تقريب إلى رقمين بعد الفاصلة
print(math.cos(0))           # جيب التمام
```

الناتج

```
12.0
3.141592653589793
3.14
1.0
```

من أرض المصنع

القدرة الفعلية لمحرّك ثلاثي الطور تُحسب بالعلاقة: $P = \sqrt{3} \times U \times I \times \cos\varphi$. لنحسبها لمحرّك يعمل تحت 400 فولط ويسحب 32 أمبير بمعامل قدرة 0.85:

Python

```python
import math
U = 400
I = 32
cos_phi = 0.85
P = math.sqrt(3) * U * I * cos_phi
print("القدرة =", round(P / 1000, 2), "kW")
```

الناتج

```
القدرة = 18.84 kW
```

خلاصة الفصل

- `+ - * /` للحساب العادي، و `//` للقسمة الصحيحة، و `%` للباقي، و `**` للأُس.
- الأقواس أولًا، ثم الأُس، ثم الضرب والقسمة، ثم الجمع والطرح.
- `+=` و `-=` اختصارات مفيدة.
- المقارنات `== != > <= >=` تعطي `True` أو `False`.
- `and` و `or` و `not` لربط الشروط، و `math` للعمليات المتقدمة.

تحدّي الفصل

مصباح قدرته 60 واط يعمل 5 ساعات يوميًّا. احسب استهلاكه الشهري (30 يومًا) بالكيلوواط ساعي، ثم تكلفته إذا كان سعر الكيلوواط ساعي 4.2 دينار. اطبع النتيجتين مقرّبتين إلى رقمين بعد الفاصلة.

الفصل

04

النصوص

الحروف والكلمات والجمل: كيف نصنعها ونقصّها ونلصقها ونحوّلها

في هذا الفصل ستتعلّم

- إنشاء النصوص بعلامات التنصيص
- سلاسل f السحرية لتنسيق الجمل
- قصّ أجزاء من النص
- لصق النصوص وتكرارها
- الوصول إلى الحروف بالفهرس
- أهم دوال النصوص



النص بالنسبة لي قطار من الحروف، وكل عربة لها رقم يبدأ من الصفر!

4.1 إنشاء النصوص

الفهرس

النص (`str`) هو أيّ سلسلة من الحروف بين علامتَي تنصيص مزدوجتين `"` أو مفردتين `'` ، والاثنتان سواء:

Python

```python
city = "الجزائر"
lang = 'Python'
message = "قال المهندس: 'المحرّك يعمل'"
```

وللنص الطويل الممتدّ على عدّة أسطر نستعمل ثلاث علامات تنصيص:

Python

```python
report = """تقرير الصيانة
المحرّك: M-12
الحالة: جيدة"""
print(report)
```

الناتج

```
تقرير الصيانة
المحرّك: M-12
الحالة: جيدة
```

4.2 اللصق والتكرار

الفهرس

العلامة `+` تلصق النصوص ببعضها، والعلامة `*` تكرّرها:

Python

```python
first = "سفيان"
last = "مرابطي"
full = first + " " + last
print(full)
print("=" * 20)
print("ها" * 3)
```

الناتج

```
سفيان مرابطي
====================
هاهاها
```

انتبه!

لا يمكنك لصق نص بعدد مباشرة: `"العمر: " + 30` تعطي خطأ `TypeError`. حوّل العدد أولًا: `"العمر: " + str(30)`، أو استعمل الطريقة الأجمل في القسم التالي.

4.3 سلاسل f: أجمل طريقة لتركيب الجمل

الفهرس

ضع الحرف `f` قبل علامة التنصيص، ثم اكتب أيّ متغيّر بين قوسين معقوفين `{}` داخل النص:

Python

```python
name = "خالد"
age = 30
salary = 85000.456
print(f"اسمي {name} وعمري {age} سنة")
print(f"بعد 5 سنوات سيكون عمري {age + 5}")
print(f"الراتب: {salary:.2f} دينار")
print(f"الراتب: {salary:,.0f} دينار")
```

الناتج

```
اسمي خالد وعمري 30 سنة
بعد 5 سنوات سيكون عمري 35
الراتب: 85000.46 دينار
الراتب: 85,000 دينار
```

التنسيق `:.2f` معناه «رقمان بعد الفاصلة»، و `:,` يضع فاصلًا بين كل ثلاثة أرقام لتسهيل القراءة.

نصيحة ذهبية

اعتمد سلاسل f في كل برامجك، فهي أوضح وأسرع من الطرق القديمة. يمكنك أيضًا أن تكتب `{x=}` لترى اسم المتغيّر مع قيمته، وهذا مفيد جدًّا عند البحث عن الأخطاء: `print(f"{age=}")` تطبع `age=30`.

4.4 الفهرسة: كل حرف له رقم

الفهرس

كل حرف في النص له رقم يسمّى **الفهرس** (index)، **والعدّ يبدأ من الصفر لا من الواحد**:

الحرف	P	y	t	h	o	n
الفهرس الموجب	0	1	2	3	4	5
الفهرس السالب	6-	5-	4-	3-	2-	1-

Python
python

```python
word = "Python"
print(word[0])
print(word[3])
print(word[-1])
print(len(word))
```

الناتج

```
P
h
n
6
```

الفهرس السالب يعدّ من النهاية: `-1` هو آخر حرف دائمًا مهما كان طول النص. والدالة `len()` تعطي عدد الحروف.

4.5 القصّ: أخذ جزء من النص

الفهرس

نكتب `[البداية:النهاية]` لأخذ قطعة من النص، **والنهاية غير محسوبة**:

Python
python

```python
code = "MOTOR-2026-A"
print(code[0:5])
print(code[6:10])
print(code[:5])
print(code[-1:])
print(code[::-1])
```

الناتج

```
MOTOR
2026
MOTOR
A
A-6202-ROTOM
```

معلومة

`[::-1]` حيلة شهيرة تقلب النص كله. وصيغة القصّ الكاملة هي `[البداية:النهاية:الخطوة]`، فـ `"123456"[::2]` تعطي `"135"`.

4.6 دوال النصوص المفيدة

الفهرس

للنصوص «دوال خاصة» (تسمّى توابع أو methods) نستدعيها بنقطة بعد النص:

الدالة	ماذا تفعل	المثال	النتيجة
`upper()`	حروف كبيرة	`"abc".upper()`	`"ABC"`
`lower()`	حروف صغيرة	`"ABC".lower()`	`"abc"`
`strip()`	تحذف المسافات من الطرفين	`" hi ".strip()`	`"hi"`
`replace(a, b)`	تستبدل a بـ b	`"1,5".replace(",", ".")`	`"1.5"`
`split(s)`	تقسّم النص إلى قائمة	`"a-b".split("-")`	`['a', 'b']`
`startswith(s)`	هل يبدأ بـ s؟	`"M12".startswith("M")`	`True`
`count(s)`	كم مرة تكرّر s؟	`"banana".count("a")`	`3`
`find(s)`	أين يوجد s؟	`"hello".find("l")`	`2`

وللبحث عن كلمة داخل نص نستعمل الكلمة `in`:

Python

```python
alarm = "خطأ: ارتفاع حرارة المحرّك"
print("حرارة" in alarm)
print("ضغط" in alarm)
```

الناتج

```
True
False
```

من أرض المصنع

تحويل قراءة مكتوبة بفاصلة أوروبية إلى عدد صالح للحساب، وهي مشكلة تتكرّر مع ملفات الأجهزة:

Python

```python
reading = " 231,7 V "
value = float(reading.strip().replace(" V", "").replace(",", "."))
print(value * 2)
```

الناتج

```
463.4
```

انتبه!

النصوص في بايثون **لا تتغيّر** (immutable). الدوال مثل `upper()` لا تغيّر النص الأصلي بل تعطيك نصًّا جديدًا. لذا اكتب: `name = name.upper()` إن أردت حفظ النتيجة.

خلاصة الفصل

- النص بين `" "` أو `' '`، والنص متعدّد الأسطر بين `""" """` .
- `+` تلصق و `*` تكرّر، و `len()` تعطي الطول.
- `f"...{x}..."` أفضل طريقة لتركيب الجمل، و `:.2f` لتحديد الأرقام العشرية.
- الفهرس يبدأ من 0، و `-1` آخر حرف، و `[a:b]` للقصّ.
- `upper` و `strip` و `replace` و `split` و `in` من أكثر الأدوات استعمالًا.

تحدّي الفصل

لديك رمز قطعة غيار `"pump-0457-dz"` . اكتب برنامجًا يطبع: نوع القطعة بحروف كبيرة (`PUMP`)، ورقمها كعدد صحيح (`457`)، ورمز البلد (`DZ`)، ثم جملة كاملة بسلسلة f: «القطعة PUMP رقم 457 من DZ».

الفصل

05

الحوار مع المستخدم

برامج تسألك وتنتظر جوابك، ثم ترد عليك

في هذا الفصل ستتعلّم

- قراءة ما يكتبه المستخدم بالدالة input
- خيارات print الخفيّة sep و end
- بناء أوّل برنامج تفاعلي حقيقي
- تحويل المدخلات إلى أعداد
- تنسيق الجداول والأرقام



حتى الآن كنت أتكلّم وحدي... الآن جاء دورك لتكلّمني!

5.1 الدالة input

الفهرس

تعرض `input()` سؤالًا على الشاشة، ثم **تنتظر** حتى يكتب المستخدم شيئًا ويضغط `Enter`، وتعيد ما كتبه:

```python
name = input("ما اسمك؟ ")
print(f"أهلًا وسهلًا يا {name}!")
```

الناتج

```
ما اسمك؟ مريم
أهلًا وسهلًا يا مريم!
```

> **نصيحة ذهبية**
>
> اترك مسافة في آخر نص السؤال: `" ما اسمك؟ "`، حتى لا يلتصق جواب المستخدم بعلامة الاستفهام.

5.2 انتبه: input تعيد نصًّا دائمًا!

الفهرس

هذا أشهر فخّ في بايثون. حتى لو كتب المستخدم رقمًا، فإن `input` تعيده **نصًّا**:

```python
a = input("العدد الأول: ")
b = input("العدد الثاني: ")
print(a + b)
```

الناتج

```
العدد الأول: 5
العدد الثاني: 3
53
```

النتيجة `53` لا `8`، لأن بايثون لصقت النصّين `"5"` و `"3"` بدل أن تجمع العددين! الحل: حوّل المدخلات إلى أعداد فورًا:

```python
a = float(input("العدد الأول: "))
b = float(input("العدد الثاني: "))
print(f"المجموع = {a + b}")
```

```
العدد الأول: 5
العدد الثاني: 3
المجموع = 8.0
```

انتبه!

إذا كتب المستخدم حروفًا بدل الأرقام، فإن `float()` تعطي خطأ `ValueError` ويتوقّف البرنامج. في الفصل الثاني عشر ستتعلّم كيف تلتقط هذا الخطأ بلطف وتطلب منه إعادة الإدخال.

5.3 أسرار الدالة print

الفهرس

للدالة `print` خياران مفيدان: `sep` لتحديد الفاصل بين القيم (الافتراضي مسافة)، و `end` لتحديد ما يُطبع في النهاية (الافتراضي سطر جديد):

Python

```python
print("2026", "10", "26", sep="/")
print("جارٍ التحميل", end="")
print("...", end="")
print(" تمّ!")
```

```
2026/10/26
جارٍ التحميل ...تمّ!
```

الرموز الخاصة

الرمز	معناه
`\n`	سطر جديد
`\t`	مسافة جدولة (Tab)
`\\`	الخط المائل نفسه
`\"`	علامة تنصيص داخل النص

Python

```python
print("السطر الأول\nالسطر الثاني")
```

```
السطر الأول
السطر الثاني
```

5.4 تنسيق الأرقام والجداول

الفهرس

داخل سلاسل f يمكنك التحكّم في عرض كل قيمة ومحاذاتها:

```python
print(f"{'المعدّة':<12}{'القدرة':>8}")
print(f"{'ضاغط هواء':<12}{75.5:>8.1f}")
print(f"{'مروحة':<12}{11:>8.1f}")
print(f"{'كسّارة':<12}{250:>8.1f}")
print(f"المردود: {0.873:.1%}")
```

```
المعدّة          القدرة
ضاغط هواء         75.5
مروحة              11.0
كسّارة            250.0
المردود: 87.3%
```

الصيغة	المعنى
`:<12`	محاذاة لليسار في عرض 12 خانة
`:>8`	محاذاة لليمين في عرض 8 خانات
`:^10`	في الوسط
`:.1f`	رقم واحد بعد الفاصلة
`:.1%`	نسبة مئوية

5.5 مشروع صغير: حاسبة قانون أوم

الفهرس

لنجمع كل ما تعلّمناه في برنامج حقيقي يحسب التيار والقدرة في مقاومة كهربائية:

ohm.py — python

```python
print("⚡ حاسبة قانون أوم ⚡")
print("-" * 24)

U = float(input("الجهد (فولط): "))
R = float(input("المقاومة (أوم): "))

I = U / R          # قانون أوم
P = U * I          # القدرة

print("-" * 24)
print(f"التيار  I = {I:.3f} A")
print(f"القدرة  P = {P:.1f} W")
```

الناتج

```
⚡ حاسبة قانون أوم ⚡
------------------------
الجهد (فولط): 220
المقاومة (أوم): 47
------------------------
التيار  I = 4.681 A
القدرة  P = 1029.8 W
```



بايثو يقول:

مبروك! لقد كتبت برنامجًا يتحاور مع البشر. لكن جرّب أن تُدخل مقاومة تساوي صفرًا... سترى خطأ `ZeroDivisionError`. سنتعلّم كيف نمنعه في الفصل القادم!

خلاصة الفصل

- `input("سؤال")` تقرأ ما يكتبه المستخدم، وتعيده **نصًّا** دائمًا.
- حوّل الأعداد فورًا: `float(input(...))` أو `int(input(...))`.
- `sep` و `end` تتحكّمان في شكل الطباعة، و `\n` سطر جديد.
- `<:` و `>:` و `:.2f` و `:.1%` لتنسيق الجداول والأرقام.

تحدّي الفصل

اكتب برنامجًا يسأل عن طول غرفة وعرضها بالمتر، وعن سعر المتر المربّع من البلاط، ثم يطبع فاتورة صغيرة منسّقة فيها: المساحة، والسعر الإجمالي، والسعر مع إضافة 10% احتياطًا للكسر.

الفصل

06

الشروط واتخاذ القرار

كيف يختار البرنامج طريقه: إذا... وإلّا...

في هذا الفصل ستتعلّم

- الجملة الشرطية if
- else و elif لتعدّد الخيارات
- الشرط في سطر واحد
- أهمية الإزاحة في بايثون
- الشروط المركّبة والمتداخلة
- الجملة match للحالات المتعدّدة



البرنامج بلا شروط مثل سيارة بلا مقود: تمشي... لكن إلى الأمام فقط!

6.1 الجملة if

الفهرس

نستعمل `if` (إذا) لنقول للبرنامج: «نفّذ هذه الأسطر **فقط** إذا كان الشرط صحيحًا»:

Python

```python
temperature = 92

if temperature > 85:
    print("⚠ إنذار: الحرارة مرتفعة!")
    print("أوقف المحرّك فورًا")

print("انتهى الفحص")
```

الناتج

```
⚠ إنذار: الحرارة مرتفعة!
أوقف المحرّك فورًا
انتهى الفحص
```

لو كانت الحرارة 70 مثلًا، لتجاوزت بايثون السطرين المُزاحين، وطبعت «انتهى الفحص» فقط.

6.2 الإزاحة: قانون بايثون الذهبي

الفهرس

لاحظ أمرين في المثال السابق:

1. سطر `if` ينتهي بنقطتين `:` .
2. الأسطر التابعة للشرط **مُزاحة** إلى الداخل بأربع مسافات.

في أغلب اللغات الأخرى تُحدَّد الكتل بالأقواس `{ }` ، أما في بايثون فالإزاحة **إجبارية** وهي التي تحدّد ما ينتمي إلى الشرط وما لا ينتمي إليه.

> **انتبه!**
>
> لا تخلط أبدًا بين المسافات وزرّ `Tab` في الملف نفسه، وإلا ظهر لك الخطأ `IndentationError` . لحسن الحظ، IDLE و VS Code يحوّلان زرّ Tab إلى أربع مسافات تلقائيًّا.

6.3 else: وإلّا...

الفهرس

```python
age = int(input("كم عمرك؟ "))

if age ⩾ 18:
    print("يمكنك الحصول على رخصة السياقة")
else:
    print(f"انتظر {18 - age} سنوات أخرى")
```

الناتج

```
كم عمرك؟ 15
انتظر 3 سنوات أخرى
```

6.4 elif: عندما تكثر الخيارات

الفهرس

`elif` اختصار لـ «else if»، أي «وإلّا إذا». تفحص بايثون الشروط بالترتيب، وتنفّذ **أوّل** شرط صحيح فقط ثم تتجاهل الباقي:

```python
grade = float(input("أدخل المعدّل: "))

if grade ⩾ 16:
    print("ممتاز 🌟")
elif grade ⩾ 14:
    print("جيد جدًّا")
elif grade ⩾ 12:
    print("جيد")
elif grade ⩾ 10:
    print("مقبول")
else:
    print("راسب، لا تيأس!")
```

الناتج

```
أدخل المعدّل: 14.5
جيد جدًّا
```

> **معلومة**
>
> لماذا لم تطبع بايثون «جيد» و«مقبول» أيضًا مع أن 14.5 أكبر من 12 و 10؟ لأنها توقّفت عند أوّل شرط صحيح. لذلك **ترتيب الشروط مهم**: ابدأ دائمًا بالأكثر تشدّدًا.

6.5 الشروط المركّبة

الفهرس

نجمع عدّة شروط بـ `and` و `or` التي تعرّفنا عليها في الفصل الثالث:

Python

```python
voltage = 395
current = 48

if voltage < 360 or voltage > 440:
    print("الجهد خارج المجال المسموح")
elif current > 45 and voltage > 380:
    print("حمولة زائدة")
else:
    print("✅ كل شيء على ما يرام")
```

الناتج

```
حمولة زائدة
```

الشروط المتداخلة

يمكن أن نضع `if` داخل `if`، مع زيادة الإزاحة في كل مستوى:

Python

```python
running = True
vibration = 7.8   # مم/ثانية

if running:
    if vibration > 7.1:
        print("اهتزاز خطير: برمج صيانة عاجلة")
    else:
        print("الاهتزاز طبيعي")
else:
    print("المعدّة متوقّفة")
```

الناتج

```
اهتزاز خطير: برمج صيانة عاجلة
```

نصيحة ذهبية

إذا وجدت نفسك تكتب ثلاثة مستويات من `if` المتداخلة أو أكثر، فتوقّف وفكّر: غالبًا يمكنك تبسيطها بـ `and` أو `elif` . الكود «المسطّح» أسهل قراءة.

6.6 الشرط في سطر واحد

الفهرس

للقرارات البسيطة يمكن كتابة الشرط في سطر واحد:

Python

```python
n = 7
kind = "زوجي" if n % 2 == 0 else "فردي"
print(f"العدد {n} {kind}")
```

الناتج

```
العدد 7 فردي
```

6.7 الجملة match

الفهرس

عندما نقارن متغيّرًا واحدًا بقيم كثيرة، تكون `match` أنظف من سلسلة طويلة من `elif` (متوفّرة منذ بايثون 3.10):

Python

```python
code = input("رمز الحالة: ")

match code:
    case "R":
        print("🟢 يعمل")
    case "S":
        print("🔴 متوقّف")
    case "M":
        print("🔧 في الصيانة")
    case _:
        print("رمز غير معروف")
```

```
رمز الحالة: M
🔧 في الصيانة
```

الحالة `_` تعني «أيّ شيء آخر»، وهي تشبه `else` .

من أرض المصنع

نظام إنذار بسيط لفرن إسمنت، بثلاثة مستويات حسب حرارة الغلاف الخارجي:

```python
shell_temp = float(input("حرارة غلاف الفرن (°C): "))

if shell_temp ⩾ 400:
    level = "🔴 خطر: احتمال سقوط البطانة الحرارية"
elif shell_temp ⩾ 330:
    level = "🟠 تحذير: راقب المنطقة عن قرب"
else:
    level = "🟢 عادي"

print(level)
```

```
حرارة غلاف الفرن (°C): 352
🟠 تحذير: راقب المنطقة عن قرب
```

خلاصة الفصل

- `if شرط:` ثم أسطر مُزاحة بأربع مسافات.
- `elif` لخيارات إضافية، و `else` لكل ما تبقّى، وتُنفَّذ أوّل حالة صحيحة فقط.
- الإزاحة إجبارية في بايثون وهي التي تحدّد الكتل.
- `x if شرط else y` للقرار في سطر واحد، و `match` للحالات الكثيرة.

تحدّي الفصل

اكتب برنامجًا يحسب فاتورة الكهرباء بنظام الشرائح: أوّل 125 كيلوواط ساعي بسعر 1.8 دينار، وما بين 126 و 250 بسعر 4.2، وما زاد على 250 بسعر 5.5. اطلب الاستهلاك من المستخدم واطبع المبلغ الإجمالي.

الفصل

07

الحلقات والتكرار

دع الحاسوب يكرّر العمل الممل ألف مرة دون أن يشتكي

في هذا الفصل ستتعلّم

- الحلقة for والدالة range
- الحلقة while وشرط التوقّف
- الحلقات المتداخلة
- المرور على حروف نص
- التحكّم بـ break و continue
- لعبة «خمّن الرقم»



أنا أحب الحلقات! أدور وأدور وأدور... حتى تقول لي: توقّف!

7.1 لماذا نحتاج الحلقات؟

الفهرس

تخيّل أنك تريد طباعة الأعداد من 1 إلى 100. هل ستكتب `print` مئة مرة؟ بالطبع لا! الحلقة (loop) تكرّر مجموعة من الأوامر عددًا من المرات نحدّده نحن.

7.2 الحلقة for مع range

الفهرس

Python

```python
for i in range(5):
    print("مرحبًا", i)
```

الناتج

```
مرحبًا 0
مرحبًا 1
مرحبًا 2
مرحبًا 3
مرحبًا 4
```

`range(5)` تولّد الأعداد من 0 إلى 4 (**خمسة أعداد، والخمسة نفسها غير محسوبة**). وفي كل دورة، يأخذ المتغيّر `i` العدد التالي ثم تُنفَّذ الأسطر المُزاحة.

الصيغة	الأعداد المولّدة
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 20, 5)`	0, 5, 10, 15
`range(10, 0, -2)`	10, 8, 6, 4, 2

مثال: جدول الضرب

Python

```python
n = 7
for i in range(1, 11):
    print(f"{n} × {i:>2} = {n * i:>3}")
```

الناتج

```
7 ×  1 =  7
7 ×  2 = 14
7 ×  3 = 21
7 ×  4 = 28
7 ×  5 = 35
7 ×  6 = 42
7 ×  7 = 49
7 ×  8 = 56
7 ×  9 = 63
7 × 10 = 70
```

مثال: المجموع التراكمي

نمط برمجي ستستعمله كثيرًا: نبدأ بمتغيّر قيمته صفر، ونضيف إليه في كل دورة:

Python

```python
total = 0
for k in range(1, 101):
    total += k
print("مجموع الأعداد من 1 إلى 100 =", total)
```

الناتج

```
مجموع الأعداد من 1 إلى 100 = 5050
```

7.3 المرور على النصوص

الفهرس

يمكن لـ `for` أن تمرّ على أي «مجموعة» من العناصر، لا الأعداد فقط. مع النص، تمرّ على حروفه واحدًا واحدًا:

Python

```python
for letter in "Python":
    print(letter, end=" ")
```

الناتج

```
P y t h o n
```

7.4 الحلقة while

الفهرس

تستمر `while` (ما دام) في التكرار **ما دام الشرط صحيحًا**. نستعملها حين لا نعرف مسبقًا عدد الدورات:

Python

```python
level = 100   # مستوى الخزّان %
hours = 0
while level > 20:
    level -= 15
    hours += 1
print(f"بعد {hours} ساعات يصل المستوى إلى {level}% فنعيد الملء")
```

الناتج

```
بعد 6 ساعات يصل المستوى إلى 10% فنعيد الملء
```

انتبه!

إذا لم يتغيّر شيء داخل `while` يجعل الشرط خاطئًا يومًا ما، فستدور الحلقة **إلى الأبد**! وإذا حدث لك ذلك، اضغط `Ctrl` `C` لإيقاف البرنامج.

7.5 continue g break

الفهرس

- `break` : اخرج من الحلقة **فورًا**.
- `continue` : تجاوز بقية هذه الدورة وانتقل إلى **الدورة التالية**.

Python

```python
for n in range(1, 11):
    if n % 2 == 0:
        continue      # تجاهل الأعداد الزوجية
    if n > 7:
        break         # توقّف بعد 7
    print(n, end=" ")
```

الناتج

```
1 3 5 7
```

نمط مفيد جدًّا: حلقة «لا نهائية» `while True` نخرج منها بـ `break` حين يكتب المستخدم كلمة معيّنة:

```python
while True:
    cmd = input("أمر (اكتب خروج للإنهاء): ")
    if cmd == "خروج":
        break
    print("نفّذت:", cmd)
print("مع السلامة 👋")
```

7.6 الحلقات المتداخلة

الفهرس

حلقة داخل حلقة: الداخلية تكتمل كلها في كل دورة من الخارجية:

```python
for row in range(1, 5):
    for col in range(row):
        print("⭐", end="")
    print()
```

الناتج

⭐
⭐⭐
⭐⭐⭐
⭐⭐⭐⭐

7.7 مشروع: لعبة «خمّن الرقم»

الفهرس

يختار الحاسوب رقمًا عشوائيًّا، وعليك أن تخمّنه بأقل عدد من المحاولات. نستعمل المكتبة `random` :

guess.py

```python
import random

secret = random.randint(1, 100)
tries = 0
print("🎲 اخترت رقمًا بين 1 و 100، هل تستطيع تخمينه؟")

while True:
    guess = int(input("تخمينك: "))
    tries += 1
    if guess < secret:
        print("⬆ أكبر من ذلك")
    elif guess > secret:
        print("⬇ أصغر من ذلك")
    else:
        print(f"🎉 أحسنت! الرقم هو {secret}، وجدته في {tries} محاولات")
        break
```

الناتج

```
🎲 اخترت رقمًا بين 1 و 100، هل تستطيع تخمينه؟
تخمينك: 50
⬆ أكبر من ذلك
تخمينك: 75
⬇ أصغر من ذلك
تخمينك: 62
🎉 أحسنت! الرقم هو 62، وجدته في 3 محاولات
```

بايثو يقول:

سرّ صغير: إذا بدأت دائمًا من المنتصف ثم قسمت المجال إلى نصفين في كل مرة، فستجد أيّ رقم بين 1 و 100 في 7 محاولات على الأكثر. هذه فكرة «البحث الثنائي» الشهيرة في علوم الحاسوب!



خلاصة الفصل

- `for x in range(...)` عندما نعرف عدد الدورات، و `range(a, b)` لا تشمل b.
- `for` تمرّ على أيّ مجموعة عناصر: أعداد، حروف، قوائم...
- `while شرط:` تكرّر ما دام الشرط صحيحًا، فاحذر الحلقة اللانهائية.
- `break` للخروج فورًا، و `continue` للانتقال إلى الدورة التالية.

تحدّي الفصل

اكتب برنامجًا يطلب من المستخدم إدخال قراءات حرارة واحدة تلو الأخرى، ويتوقّف عندما يكتب `0` . في النهاية يطبع: عدد القراءات، ومتوسّطها، وأعلى قراءة.

الفصل

08

القوائم والصفوف

كيف نحفظ مئات القيم في متغيّر واحد ونتحكّم فيها بسهولة

في هذا الفصل ستتعلّم

- إنشاء القوائم والوصول إلى عناصرها
- إضافة العناصر وحذفها وتعديلها
- الترتيب والبحث والإحصاء
- المرور على القوائم بالحلقات
- فهم القوائم (List Comprehension)
- الصفوف tuple والتفكيك



القائمة مثل قطار طويل، يمكنك أن تضيف إليه عربات أو تنزعها متى شئت!

8.1 ما هي القائمة؟

الفهرس

لو أردت حفظ قراءات 30 يومًا، فلن تنشئ 30 متغيّرًا! **القائمة** (list) تحفظ مجموعة من القيم بالترتيب، بين قوسين مربّعين `[ ]` :

```python
temps = [21.5, 23.0, 19.8, 25.4, 22.1]
names = ["علي", "سارة", "يوسف"]
mixed = [1, "نص", True, 3.14]
empty = []
print(temps)
print(len(names))
```

الناتج

```
[21.5, 23.0, 19.8, 25.4, 22.1]
3
```

8.2 الوصول إلى العناصر

الفهرس

تعمل الفهرسة والقصّ تمامًا كما في النصوص، والعدّ يبدأ من الصفر:

```python
fruits = ["تفاح", "موز", "برتقال", "عنب", "مشمش"]
print(fruits[0])
print(fruits[-1])
print(fruits[1:3])
```

الناتج

```
تفاح
مشمش
['موز', 'برتقال']
```

لكن على عكس النصوص، **القوائم قابلة للتعديل**:

```python
fruits[1] = "فراولة"
print(fruits)
```

الناتج

```
['تفاح', 'فراولة', 'برتقال', 'عنب', 'مشمش']
```

8.3 الإضافة والحذف

الفهرس

الدالة	ماذا تفعل
`append(x)`	تضيف x في **آخر** القائمة
`insert(i, x)`	تضيف x في الموضع i
`remove(x)`	تحذف أول عنصر قيمته x
`pop()`	تحذف آخر عنصر وتعيده
`pop(i)`	تحذف العنصر الذي في الموضع i وتعيده
`clear()`	تفرّغ القائمة كلها

Python

```python
tasks = ["فحص الزيت", "تنظيف المرشّح"]
tasks.append("شدّ السيور")
tasks.insert(0, "إيقاف الآلة")
print(tasks)
done = tasks.pop(0)
print("أُنجزت:", done)
tasks.remove("تنظيف المرشّح")
print(tasks)
```

الناتج

```
['إيقاف الآلة', 'فحص الزيت', 'تنظيف المرشّح', 'شدّ السيور']
أُنجزت: إيقاف الآلة
['فحص الزيت', 'شدّ السيور']
```

8.4 الترتيب والبحث والإحصاء

الفهرس

```python
power = [75, 11, 250, 37, 160, 4]
print("المجموع:", sum(power))
print("الأكبر:", max(power))
print("الأصغر:", min(power))
print("المتوسّط:", sum(power) / len(power))
print("هل 37 موجود؟", 37 in power)
print("موضع 250:", power.index(250))

power.sort()
print("تصاعديًّا:", power)
power.sort(reverse=True)
print("تنازليًّا:", power)
```

الناتج

```
المجموع: 537
الأكبر: 250
الأصغر: 4
المتوسّط: 89.5
هل 37 موجود؟ True
موضع 250: 2
تصاعديًّا: [4, 11, 37, 75, 160, 250]
تنازليًّا: [250, 160, 75, 37, 11, 4]
```

نصيحة ذهبية

`sort()` تغيّر القائمة الأصلية. إذا أردت نسخة مرتّبة مع الحفاظ على الأصل، استعمل `sorted(power)` التي تعيد قائمة جديدة.

8.5 المرور على القوائم

الفهرس

الحلقة `for` والقوائم صديقان حميمان:

```python
readings = [221, 229, 218, 241, 225]
for r in readings:
    if r > 230:
        print(f"{r} V ⚠ مرتفع")
    else:
        print(f"{r} V ✓")
```

الناتج

```
221 V ✓
229 V ✓
218 V ✓
241 V ⚠ مرتفع
225 V ✓
```

وإذا احتجت الرقم التسلسلي مع القيمة، فاستعمل `enumerate`:

```python
machines = ["كسّارة", "طاحونة", "فرن"]
for i, m in enumerate(machines, start=1):
    print(i, m)
```

الناتج

```
1 كسّارة
2 طاحونة
3 فرن
```

8.6 فهم القوائم: سطر واحد يغني عن أربعة

الفهرس

List comprehension طريقة مختصرة وأنيقة لإنشاء قائمة من قائمة أخرى:

```python
celsius = [0, 25, 37, 100]
fahrenheit = [c * 9 / 5 + 32 for c in celsius]
print(fahrenheit)

hot = [c for c in celsius if c > 30]
print(hot)
```

الناتج
```
[32.0, 77.0, 98.6, 212.0]
[37, 100]
```

اقرأها هكذا: «أعطني `c * 9 / 5 + 32` لكل `c` في `celsius`». والجزء `if` في المثال الثاني يرشّح العناصر.

> **انتبه!**
> النسخ الخاطئ! الكتابة `b = a` **لا تنسخ** القائمة، بل تجعل الاسمين يشيران إلى القائمة نفسها، فكل تعديل على `b` يظهر في `a`. للنسخ الحقيقي اكتب `b = a.copy()`.

8.7 الصفوف tuple

الفهرس

الصف يشبه القائمة لكنه يُكتب بأقواس عادية `( )` و**لا يمكن تعديله** بعد إنشائه. نستعمله للقيم الثابتة التي يجب ألّا تتغيّر:

Python
```python
point = (36.75, 3.06)        # إحداثيات
rgb = (255, 212, 59)         # لون
print(point[0])
```

الناتج
```
36.75
```

التفكيك (unpacking)

يمكن «تفريغ» عناصر الصف أو القائمة في متغيّرات مباشرة:

Python
```python
lat, lon = point
r, g, b = rgb
print(f"خط العرض {lat}، خط الطول {lon}")
```

الناتج
```
خط العرض 36.75، خط الطول 3.06
```

من أرض المصنع

تحليل سريع لقراءات تيار محرّك خلال وردية عمل:

Python

```python
amps = [31.2, 32.8, 30.9, 45.1, 33.0, 31.7, 46.3, 32.2]
limit = 40
over = [a for a in amps if a > limit]
print(f"المتوسّط: {sum(amps) / len(amps):.1f} A")
print(f"تجاوزات الحدّ: {len(over)} مرّات ← {over}")
```

الناتج

```
المتوسّط: 35.4 A
تجاوزات الحدّ: 2 مرّات ← [45.1, 46.3]
```

خلاصة الفصل

- القائمة `[ ]` مرتّبة وقابلة للتعديل، والصف `( )` ثابت لا يتغيّر.
- `append` و `insert` و `remove` و `pop` للإضافة والحذف.
- `sum` و `max` و `min` و `len` و `sort` و `in` للإحصاء والبحث.
- `[تعبير for x in قائمة if شرط]` لصنع قوائم جديدة في سطر واحد.
- `b = a.copy()` للنسخ الحقيقي.

تحدّي الفصل

لديك قائمة بأسماء العمّال وقائمة أخرى بساعات عملهم في الأسبوع (بالترتيب نفسه). اطبع اسم كل عامل وساعاته، ثم اطبع أسماء من تجاوزوا 40 ساعة، وأخيرًا اسم صاحب أكبر عدد من الساعات. (تلميح: ابحث عن الدالة `zip`).

الفصل

09

القواميس والمجموعات

ابحث عن أيّ معلومة باسمها في لمح البصر

في هذا الفصل ستتعلّم

- فكرة المفتاح والقيمة
- إضافة البيانات وتعديلها وحذفها
- القواميس المتداخلة
- إنشاء القواميس وقراءتها بأمان
- المرور على القواميس بالحلقات
- المجموعات set وعملياتها

في القاموس لا تبحث عن الكلمة برقم صفحتها، بل باسمها. وهكذا يعمل قاموس بايثون!



9.1 ما هو القاموس؟

الفهرس

في القائمة نصل إلى العنصر برقمه: `items[3]`. لكن ماذا لو أردنا أن نصل إليه **باسمه**؟ هنا يأتي **القاموس** (dict): مجموعة من الأزواج، كل زوج فيه **مفتاح** و**قيمة**، بين قوسين معقوفين `{ }`:

Python

```python
motor = {
    "name": "مضخة التبريد",
    "power": 37,
    "voltage": 400,
    "running": True,
}
print(motor["name"])
print(motor["power"], "kW")
```

الناتج

```
مضخة التبريد
37 kW
```

معلومة

فكّر في القاموس كبطاقة تعريف: «الاسم: ...، القدرة: ...، الجهد: ...». المفاتيح غالبًا نصوص، ويجب أن تكون **فريدة**: لا يمكن أن يتكرّر المفتاح نفسه مرتين.

9.2 القراءة بأمان باستعمال get

الفهرس

إذا طلبت مفتاحًا غير موجود بالأقواس المربّعة، يتوقّف البرنامج بخطأ `KeyError`. الدالة `get` أكثر أمانًا، فهي تعيد `None` أو قيمة احتياطية تحدّدها أنت:

Python

```python
print(motor.get("speed"))
print(motor.get("speed", 1480))
```

الناتج

```
None
1480
```

9.3 الإضافة والتعديل والحذف

الفهرس

```python
motor["speed"] = 1480          # إضافة مفتاح جديد
motor["power"] = 45            # تعديل قيمة موجودة
del motor["running"]           # حذف مفتاح
print(motor)
print("voltage" in motor)      # هل المفتاح موجود؟
```

الناتج

```
{'name': 'مضخة التبريد', 'power': 45, 'voltage': 400, 'speed': 1480}
True
```

9.4 المرور على القواميس

الفهرس

الدالة	تعطيك
`keys()`	المفاتيح فقط
`values()`	القيم فقط
`items()`	الأزواج (مفتاح، قيمة)

```python
stock = {"محامل": 24, "سيور": 6, "مرشّحات": 0, "زيت (لتر)": 120}

for item, qty in stock.items():
    flag = "❌ نفد" if qty == 0 else ""
    print(f"{item:<12} {qty:>4} {flag}")

print("مجموع القطع:", sum(stock.values()))
```

الناتج

```
محامل          24
سيور            6
مرشّحات         0 ❌ نفد
زيت (لتر)     120
مجموع القطع: 150
```

مثال كلاسيكي: عدّ التكرارات

```python
text = "ضغط حرارة ضغط اهتزاز ضغط حرارة"
count = {}
for word in text.split():
    count[word] = count.get(word, 0) + 1
print(count)
```

الناتج

```
{'ضغط': 3, 'حرارة': 2, 'اهتزاز': 1}
```

نصيحة ذهبية

هذا النمط `d[k] = d.get(k, 0) + 1` من أشهر الأنماط في بايثون. احفظه، فستحتاجه لعدّ الإنذارات والأعطال والكلمات وكل شيء تقريبًا.

9.5 القواميس المتداخلة

الفهرس

يمكن أن تكون القيمة نفسها قاموسًا أو قائمة، فنبني بها بيانات منظّمة تشبه قاعدة بيانات صغيرة:

```python
plant = {
    "M01": {"name": "كسّارة", "kw": 250, "hours": [7.5, 8, 6]},
    "M02": {"name": "طاحونة", "kw": 900, "hours": [8, 8, 8]},
}

for code, m in plant.items():
    energy = m["kw"] * sum(m["hours"])
    print(f"{code} {m['name']}: {energy:,} kWh")
```

الناتج

```
M01 كسّارة: 5,375.0 kWh
M02 طاحونة: 21,600 kWh
```

> **معلومة**
>
> هذا الشكل من البيانات (قواميس داخل قوائم وقواميس) هو تقريبًا صيغة **JSON** نفسها، وهي الصيغة التي تتبادل بها المواقع والتطبيقات والأجهزة الذكية بياناتها عبر الإنترنت.

9.6 المجموعات set

الفهرس

المجموعة تشبه القائمة، لكنها **لا تقبل التكرار** ولا يهمّها الترتيب. مفيدة جدًّا لحذف المكرّرات ومقارنة مجموعتين:

Python

```python
alarms = ["M01", "M03", "M01", "M02", "M03", "M01"]
unique = set(alarms)
print(unique)
print("عدد الآلات المعنيّة:", len(unique))
```

الناتج

```
{'M01', 'M02', 'M03'}
عدد الآلات المعنيّة: 3
```

(قد يختلف ترتيب العناصر عندك، فالمجموعة لا تحفظ الترتيب.)

العملية	الرمز	المعنى
الاتحاد	`a | b`	كل العناصر من الاثنتين
التقاطع	`a & b`	العناصر المشتركة فقط
الفرق	`a - b`	ما في a وليس في b

Python

```python
monday = {"M01", "M02", "M05"}
tuesday = {"M02", "M05", "M07"}
print("تعطّلت في اليومين:", sorted(monday & tuesday))
print("الإثنين فقط:", monday - tuesday)
```

الناتج

```
تعطّلت في اليومين: ['M02', 'M05']
الإثنين فقط: {'M01'}
```

خلاصة الفصل

- القاموس `{مفتاح: قيمة}` للوصول إلى البيانات بالاسم بدل الرقم.
- `d[k]` للقراءة والكتابة، و `d.get(k, احتياطي)` للقراءة الآمنة، و `del` للحذف.
- `items()` للمرور على الأزواج، و `d.get(k, 0) + 1` لعدّ التكرارات.
- المجموعة `set` لا تقبل التكرار، وتدعم الاتحاد `|` والتقاطع `&` والفرق `-` .

تحدّي الفصل

أنشئ قاموسًا لدليل هاتف ورشة الصيانة (الاسم ← الرقم). اكتب برنامجًا بقائمة خيارات: 1) بحث عن رقم، 2) إضافة شخص، 3) عرض الكل، 4) خروج، ويستمر بالعمل حتى يختار المستخدم الخروج.

الفصل

10

الدوال

اكتب الكود مرة واحدة، واستعمله ألف مرة

في هذا الفصل ستتعلّم

- لماذا نحتاج الدوال
- تعريف دالة واستدعاؤها
- المعاملات والقيم المعادة
- القيم الافتراضية والمعاملات المسمّاة
- مجال المتغيّرات المحلّي والعام
- التوثيق والدوال المختصرة lambda



الدالة مثل آلة القهوة: تضع فيها البنّ والماء، تضغط الزر... وتخرج القهوة!

10.1 لماذا الدوال؟

الفهرس

هل لاحظت أننا كنّا نكتب أحيانًا الأسطر نفسها مرّات عديدة؟ **الدالة** (function) مجموعة من الأوامر نعطيها اسمًا، ثم ننفّذها متى شئنا بذكر اسمها فقط. وقد استعملت دوالّ جاهزة منذ الفصل الأول: `print` و `input` و `len` ... والآن ستصنع دوالّك الخاصة.

فوائد الدوال:

- **عدم التكرار**: تكتب الكود مرة واحدة.
- **التنظيم**: تقسّم البرنامج الكبير إلى قطع صغيرة واضحة.
- **سهولة التصحيح**: إذا وُجد خطأ، تصلحه في مكان واحد.

10.2 تعريف الدالة واستدعاؤها

الفهرس

نعرّف الدالة بالكلمة `def` متبوعة باسمها وقوسين ونقطتين، ثم الأوامر مُزاحة:

Python

```python
def greet():
    print("السلام عليكم")
    print("مرحبًا بك في الورشة")

greet()
greet()
```

الناتج

```
السلام عليكم
مرحبًا بك في الورشة
السلام عليكم
مرحبًا بك في الورشة
```

انتبه!

تعريف الدالة **لا ينفّذها**! إنه مجرّد «وصفة». التنفيذ يحدث عند **الاستدعاء** `greet()`، ولا تنسَ القوسين، فبدونهما لن يحدث شيء.

10.3 المعاملات: نعطي الدالة معلومات

الفهرس

نضع بين القوسين **معاملات** (parameters) تستقبل القيم التي نرسلها:

```python
def greet(name, title):
    print(f"مرحبًا يا {title} {name}")

greet("أحمد", "المهندس")
greet("ليلى", "الدكتورة")
```

الناتج

```
مرحبًا يا المهندس أحمد
مرحبًا يا الدكتورة ليلى
```

10.4 return: الدالة تعيد نتيجة

الفهرس

أغلب الدوال المفيدة **تحسب شيئًا وتعيده** بالكلمة `return`، لنستعمل النتيجة كما نريد:

```python
def rectangle_area(length, width):
    return length * width

a = rectangle_area(5, 4)
b = rectangle_area(3, 2.5)
print(a, b, a + b)
```

الناتج

```
20 7.5 27.5
```

معلومة

الفرق بين `print` و `return` مهم جدًّا: `print` تعرض النتيجة على الشاشة ثم تضيع، أما `return` فتعطيك النتيجة لتحفظها في متغيّر أو تحسب بها. وعند الوصول إلى `return` تتوقّف الدالة فورًا.

يمكن للدالة أن تعيد عدّة قيم معًا (في صف):

```python
def stats(values):
    return min(values), max(values), sum(values) / len(values)

low, high, avg = stats([12, 18, 9, 21, 15])
print(f"الأدنى {low}، الأعلى {high}، المتوسّط {avg}")
```

الناتج

```
الأدنى 9، الأعلى 21، المتوسّط 15.0
```

10.5 القيم الافتراضية والمعاملات المسمّاة

الفهرس

يمكن أن نعطي المعامل قيمة افتراضية، فيصبح اختياريًّا:

```python
def energy_cost(kwh, price=4.2, tax=0.19):
    return kwh * price * (1 + tax)

print(round(energy_cost(100), 2))
print(round(energy_cost(100, price=5.5), 2))
print(round(energy_cost(kwh=100, tax=0), 2))
```

الناتج

```
499.8
654.5
420.0
```

عند الاستدعاء يمكن ذكر اسم المعامل `price=5.5`، فيصبح الكود أوضح ولا يهمّ ترتيب المعاملات.

10.6 المجال: أين يعيش المتغيّر؟

الفهرس

المتغيّر الذي يُنشأ **داخل** الدالة متغيّر محلّي: يولد عند استدعائها ويموت عند انتهائها، ولا يُرى من خارجها:

```python
def calc():
    result = 42
    return result

calc()
print(result)
```

الناتج

```
NameError: name 'result' is not defined
```

نصيحة ذهبية

القاعدة الذهبية: الدالة الجيدة **تأخذ ما تحتاجه عبر المعاملات، وتعطي نتيجتها عبر return**. تجنّب أن تعتمد الدالة على متغيّرات خارجية أو تغيّرها، فهذا يصنع أخطاء يصعب اكتشافها.

10.7 التوثيق و lambda

الفهرس

نص التوثيق (docstring)

النص الموضوع بين ثلاث علامات تنصيص في أول سطر من الدالة يشرح عملها، ويظهر عندما تكتب `help(اسم_الدالة)`:

```python
def kelvin(c):
    """تحوّل درجة الحرارة من سلسيوس إلى كلفن."""
    return c + 273.15
```

الدوال المختصرة lambda

لدالة صغيرة من سطر واحد، نستعمل `lambda`. وأشهر استعمالاتها تحديد طريقة الترتيب:

```python
machines = [("فرن", 5200), ("كسّارة", 250), ("طاحونة", 900)]
machines.sort(key=lambda m: m[1], reverse=True)
print(machines)
```

الناتج

```
[('فرن', 5200), ('طاحونة', 900), ('كسّارة', 250)]
```

من أرض المصنع

مكتبة صغيرة من الدوال الكهربائية، تكتبها مرة واحدة وتستعملها في كل برامجك:

Python

```python
import math

def power_3ph(u, i, cos_phi=0.85):
    """القدرة الفعلية لنظام ثلاثي الطور بالكيلوواط."""
    return math.sqrt(3) * u * i * cos_phi / 1000

def energy(p_kw, hours):
    """الطاقة المستهلكة بالكيلوواط ساعي."""
    return p_kw * hours

p = power_3ph(400, 32)
print(f"P = {p:.2f} kW، الطاقة في 8 ساعات = {energy(p, 8):.1f} kWh")
```

الناتج

```
P = 18.84 kW، الطاقة في 8 ساعات = 150.8 kWh
```

خلاصة الفصل

- `def name(params):` لتعريف دالة، و `name(...)` لاستدعائها.
- `return` تعيد النتيجة وتنهي الدالة، ويمكن إعادة عدّة قيم.
- القيم الافتراضية `price=4.2` تجعل المعامل اختياريًّا.
- المتغيّرات داخل الدالة محلّية لا تُرى من خارجها.
- اكتب docstring لكل دالة، واستعمل `lambda` للدوال الصغيرة جدًّا.

تحدّي الفصل

اكتب دالة `is_prime(n)` تعيد `True` إذا كان العدد أوّليًّا و `False` إن لم يكن، ثم استعملها لطباعة كل الأعداد الأوّلية بين 1 و 50.

الفصل

11

الوحدات والمكتبات

لا تخترع العجلة من جديد: آلاف الأدوات الجاهزة في انتظارك

في هذا الفصل ستتعلّم

- استيراد المكتبات بطرق مختلفة
- صناعة وحدتك الخاصة
- جولة سريعة في numpy و pandas و matplotlib
- مكتبات بايثون المدمجة الأكثر فائدة
- تثبيت المكتبات الخارجية بـ pip

لماذا تكتب 500 سطر، إذا كان أحدهم قد كتبها لك مجانًا؟ هذه هي روح بايثون!



11.1 ما هي الوحدة؟

الفهرس

الوحدة (module) ملف بايثون فيه دوال ومتغيّرات جاهزة، و**المكتبة** (library أو package) مجموعة من الوحدات. تأتي بايثون مع مئات الوحدات الجاهزة، وهذا ما يسمّيه المبرمجون «البطاريات مرفقة» (batteries included).

11.2 طرق الاستيراد

الفهرس

Python

```python
import math                    # استيراد الوحدة كاملة
print(math.sqrt(16))

from math import pi, sqrt      # استيراد أشياء محدّدة
print(pi, sqrt(25))

import datetime as dt          # استيراد مع اسم مستعار قصير
print(dt.date.today())
```

الناتج

```
4.0
3.141592653589793 5.0
2026-10-26
```

انتبه!

تجنّب `from math import *` التي تستورد كل شيء دفعة واحدة، فقد تتصادم الأسماء دون أن تنتبه، ولن تعرف من أين جاءت كل دالة.

11.3 جولة في المكتبات المدمجة

الفهرس

random: العشوائية

Python

```python
import random

print(random.randint(1, 6))                    # رمي حجر نرد
print(random.choice(["أحمر", "أخضر", "أزرق"]))
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)                          # خلط
print(cards)
```

الناتج

```
4
أخضر
[3, 1, 5, 2, 4]
```

datetime: التاريخ والوقت

Python

```python
from datetime import datetime, timedelta

now = datetime.now()
print(now.strftime("%d/%m/%Y %H:%M"))
next_service = now + timedelta(days=90)
print("الصيانة القادمة:", next_service.strftime("%d/%m/%Y"))
```

الناتج

```
26/10/2026 09:41
الصيانة القادمة: 24/01/2027
```

أدوات أخرى مفيدة

الوحدة	الاستعمال
`os`	التعامل مع الملفات والمجلّدات
`time`	الانتظار `sleep` وقياس الوقت
`statistics`	المتوسّط والوسيط والانحراف المعياري
`csv`	قراءة ملفات CSV وكتابتها
`json`	التعامل مع بيانات JSON

Python

```python
import statistics as st
data = [12.1, 12.4, 11.9, 12.8, 12.2]
print(st.mean(data), st.median(data), round(st.stdev(data), 3))
```

الناتج

```
12.28 12.2 0.342
```

الفهرس

11.4 صنع وحدتك الخاصة

أيّ ملف `.py` تكتبه هو وحدة! احفظ هذا الملف باسم `electro.py` :

electro.py

```python
"""أدوات حسابية للكهرباء الصناعية."""
import math

def power_3ph(u, i, cos_phi=0.85):
    return math.sqrt(3) * u * i * cos_phi / 1000

def current_3ph(p_kw, u, cos_phi=0.85):
    return p_kw * 1000 / (math.sqrt(3) * u * cos_phi)
```

ثم في ملف آخر **داخل المجلّد نفسه**:

main.py — python

```python
from electro import current_3ph

print(f"75 تيار محرّك kW: {current_3ph(75, 400):.1f} A")
```

الناتج

```
تيار محرّك 75 kW: 127.4 A
```

نصيحة ذهبية

لا تسمِّ ملفاتك بأسماء المكتبات الشهيرة مثل `random.py` أو `math.py`، وإلا استوردت بايثون ملفك بدل المكتبة الحقيقية وظهرت أخطاء غريبة جدًّا!

11.5 تثبيت المكتبات الخارجية بـ pip

الفهرس

يوجد أكثر من نصف مليون مكتبة مجانية على موقع `pypi.org`. نثبّتها بالأداة `pip` من نافذة الأوامر (لا من داخل بايثون):

Terminal

```
pip install numpy pandas matplotlib
```

ثم نستوردها في برامجنا كأيّ وحدة أخرى.

المكتبة	مجالها
`numpy`	الحسابات العددية والمصفوفات
`pandas`	الجداول وتحليل البيانات (مثل إكسل لكن أقوى)
`matplotlib`	الرسوم البيانية
`openpyxl`	قراءة ملفات إكسل وكتابتها
`requests`	جلب البيانات من الإنترنت
`scikit-learn`	التعلّم الآلي والذكاء الاصطناعي

11.6 لمحة: ارسم أوّل منحنى لك

الفهرس

مثال قصير يرسم منحنى استهلاك يومي، لترى بنفسك قوّة هذه المكتبات:

plot.py — python

```python
import matplotlib.pyplot as plt

hours = list(range(24))
load = [310, 300, 295, 290, 300, 340, 420, 510, 560, 575, 580, 570,
        540, 555, 570, 565, 540, 500, 470, 450, 420, 380, 350, 320]

plt.plot(hours, load, marker="o", color="#3776AB")
plt.fill_between(hours, load, alpha=0.2, color="#FFD43B")
plt.title("Daily load (kW)")
plt.xlabel("Hour")
plt.grid(True)
plt.show()
```

سيفتح هذا البرنامج نافذة فيها منحنى ملوّن يبيّن ساعات الذروة. وبأسطر قليلة إضافية مع `pandas` يمكنك قراءة آلاف القراءات من ملف إكسل وتحليلها.

من أرض المصنع

مع `pandas` تستطيع قراءة ملف إكسل فيه سجلّ عام كامل من قراءات العدّادات، ثم حساب الاستهلاك الشهري لكل ورشة في سطرين فقط. هذا ما يجعل بايثون الأداة المفضّلة لمكاتب الدراسات وأقسام الطاقة.

خلاصة الفصل

- `import x` و `from x import y` و `import x as z` ثلاث طرق للاستيراد.
- المكتبات المدمجة: `math` و `random` و `datetime` و `statistics` و `os` و `csv` ...
- أيّ ملف `.py` يمكن استيراده كوحدة.
- `pip install اسم_المكتبة` لتثبيت المكتبات الخارجية من نافذة الأوامر.

تحدّي الفصل

اكتب برنامجًا يولّد كلمة سرّ عشوائية من 12 حرفًا تحتوي على حروف كبيرة وصغيرة وأرقام ورموز. (تلميح: الوحدة `string` فيها `ascii_letters` و `digits` و `punctuation`).

الفصل

12

الملفات ومعالجة الأخطاء

احفظ بياناتك للأبد، ولا تدع أيّ خطأ يُسقط برنامجك

في هذا الفصل ستتعلّم

- كتابة الملفات النصية وقراءتها
- الإضافة إلى ملف موجود
- التعامل مع ملفات CSV
- أنواع الأخطاء الشائعة وفهمها
- التقاط الأخطاء بـ try و except
- بناء إدخال لا يتعطّل أبدًا

البرنامج المحترف ليس الذي لا يخطئ أبدًا، بل الذي يعرف ماذا يفعل عندما يخطئ.



Dr Merabti

12.1 لماذا الملفات؟

الفهرس

كل ما تحفظه في المتغيّرات يختفي عند إغلاق البرنامج. لنحفظ البيانات بشكل دائم، نكتبها في **ملف** على القرص الصلب.

12.2 الكتابة في ملف

الفهرس

Python

```python
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("تقرير الوردية الصباحية\n")
    f.write("الفرن: يعمل بشكل عادي\n")
    f.write("الطاحونة: توقّف 20 دقيقة\n")
print("تمّ الحفظ ✅")
```

الناتج

```
تمّ الحفظ ✅
```

لنفهم هذا السطر جيدًا:

- `open` تفتح الملف (وتنشئه إن لم يكن موجودًا).
- `"w"` تعني الكتابة (write)، **وتمسح المحتوى القديم** إن وُجد.
- `encoding="utf-8"` ضرورية لكتابة الحروف العربية بشكل صحيح.
- `with` تغلق الملف تلقائيًّا عند الانتهاء، حتى لو حدث خطأ.

الوضع	المعنى
`"r"`	قراءة فقط (الافتراضي)
`"w"`	كتابة جديدة (يمسح القديم)
`"a"`	إضافة في آخر الملف (append)

> **انتبه!**
> احذر من الوضع `"w"`: إذا فتحت به ملفًّا مهمًّا فسيُمسح محتواه فورًا! عندما تريد إضافة سطر إلى سجلّ موجود، استعمل `"a"`.

12.3 القراءة من ملف

الفهرس

Python

```python
with open("notes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print("•", line.strip())
```

الناتج

```
• تقرير الوردية الصباحية
• الفرن: يعمل بشكل عادي
• الطاحونة: توقّف 20 دقيقة
```

يمكن أيضًا قراءة الملف كاملًا دفعة واحدة بـ `f.read()`، أو كقائمة من الأسطر بـ `f.readlines()`.

12.4 ملفات CSV

الفهرس

ملفات CSV جداول بسيطة، كل سطر فيها صف، والأعمدة مفصولة بفواصل. يفتحها إكسل مباشرة، وتصدّرها أغلب الأجهزة وأنظمة الإشراف:

Python

```python
import csv

rows = [["machine", "kw", "hours"],
        ["crusher", 250, 7.5],
        ["mill", 900, 8],
        ["fan", 75, 24]]

with open("energy.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows(rows)

with open("energy.csv", encoding="utf-8") as f:
    for r in csv.DictReader(f):
        kwh = float(r["kw"]) * float(r["hours"])
        print(f"{r['machine']:<8} {kwh:>7.0f} kWh")
```

الناتج

```
crusher     1875 kWh
mill        7200 kWh
fan         1800 kWh
```

نصيحة ذهبية

`csv.DictReader` تحوّل كل سطر إلى قاموس مفاتيحه أسماء الأعمدة، فتكتب `r["kw"]` بدل `r[1]`، وهذا أوضح بكثير وأقلّ عرضة للخطأ.

12.5 الأخطاء الشائعة

الفهرس

كل مبرمج في العالم يرى هذه الأخطاء يوميًّا، فتعلّم أن تقرأها بهدوء:

الخطأ	سببه	مثال
`SyntaxError`	خطأ في الكتابة (قوس أو نقطتان ناقصة)	`if x > 3`
`IndentationError`	إزاحة خاطئة	سطر غير مُزاح بعد `if`
`NameError`	اسم غير معرّف	`prnt("hi")`
`TypeError`	عملية بين نوعين غير متوافقين	`"5" + 3`
`ValueError`	قيمة غير مناسبة	`int("abc")`
`ZeroDivisionError`	القسمة على صفر	`10 / 0`
`IndexError`	فهرس خارج القائمة	`[1, 2][5]`
`KeyError`	مفتاح غير موجود في القاموس	`d["xyz"]`
`FileNotFoundError`	الملف غير موجود	`open("x.txt")`

12.6 try و except: شبكة الأمان

الفهرس

نضع الكود الذي قد يفشل داخل `try`، ونحدّد ماذا نفعل عند كل خطأ في `except`:

```python
try:
    u = float(input("الجهد: "))
    r = float(input("المقاومة: "))
    print("التيار =", u / r)
except ValueError:
    print("❌ من فضلك أدخل أرقامًا فقط")
except ZeroDivisionError:
    print("❌ المقاومة لا يمكن أن تكون صفرًا")
```

الناتج

```
الجهد: 220
المقاومة: 0
❌ المقاومة لا يمكن أن تكون صفرًا
```

بدل أن يتوقّف البرنامج برسالة حمراء مخيفة، يعرض رسالة لطيفة ويواصل العمل.

الجزء	متى يُنفَّذ
`try`	دائمًا (نحاول)
`except`	عند حدوث الخطأ المحدّد
`else`	إذا لم يحدث أيّ خطأ
`finally`	في كل الحالات، خطأ أو لا

12.7 إدخال لا يتعطّل أبدًا

الفهرس

بدمج `while True` و `try` ، نصنع دالة تُلحّ على المستخدم بأدب حتى يُدخل رقمًا صحيحًا:

Python
```python
def ask_number(question):
    while True:
        try:
            return float(input(question))
        except ValueError:
            print("⚠ هذا ليس رقمًا، حاول مرة أخرى")

power = ask_number("قدرة المحرّك (kW): ")
print("تمّ:", power)
```

الناتج
```
قدرة المحرّك (kW): خمسة وسبعون
⚠ هذا ليس رقمًا، حاول مرة أخرى
قدرة المحرّك (kW): 75
تمّ: 75.0
```



بايثو يقول:

احفظ الدالة `ask_number` جيدًا، فستستعملها في المشروع النهائي. وتذكّر: لا تكتب أبدًا `except:` وحدها دون نوع الخطأ، فهي تبتلع كل الأخطاء، حتى التي لم تتوقّعها، فتختفي المشاكل الحقيقية!

من أرض المصنع

سجلّ إنذارات بسيط: كل إنذار يُضاف مع تاريخه ووقته إلى آخر الملف دون أن يمحو ما قبله:

Python
```python
from datetime import datetime

def log_alarm(message):
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("alarms.log", "a", encoding="utf-8") as f:
        f.write(f"[{stamp}] {message}\n")

log_alarm("M02 92 حرارة المحمل °C")
log_alarm("M05 8.1 اهتزاز mm/s")
```

خلاصة الفصل

- `with open(اسم, وضع, encoding="utf-8") as f:` الطريقة الصحيحة لفتح الملفات.
- `"r"` للقراءة، و `"w"` لكتابة جديدة (تمسح القديم)، و `"a"` للإضافة.
- الوحدة `csv` لجداول البيانات، و `DictReader` لقراءتها بأسماء الأعمدة.
- `try` / `except نوع_الخطأ` لالتقاط الأخطاء ومعالجتها بلطف.

تحدّي الفصل

اكتب برنامجًا يقرأ ملف `energy.csv` السابق، ويحسب الطاقة الإجمالية لكل الآلات، ثم يكتب تقريرًا في ملف جديد `report.txt` . يجب ألّا يتوقّف البرنامج إذا كان الملف غير موجود، بل يطبع رسالة واضحة.

الفصل



13

البرمجة الكائنية

اصنع «أنواعك» الخاصة: محرّكات ومضخّات وعمّال... بخصائصها وسلوكها

في هذا الفصل ستتعلّم

- الفرق بين الصنف والكائن
- الدالة البانية __init__ والكلمة self
- الطباعة الجميلة بـ __str__
- إنشاء صنف بالكلمة class
- الخصائص والتوابع
- الوراثة: صنف يرث من صنف

الصنف هو المخطّط الهندسي، والكائن هو المحرّك الحقيقي الذي صُنع منه.
مخطّط واحد... وألف محرّك!



13.1 الفكرة: الصنف والكائن

الفهرس

حتى الآن، كانت البيانات (المتغيّرات) في جهة، والعمليات عليها (الدوال) في جهة أخرى. **البرمجة الكائنية** (OOP) تجمعهما معًا في شيء واحد يسمّى **كائنًا**.

- **الصنف** (class): القالب أو المخطّط. مثلًا: «المحرّك الكهربائي» بصفة عامة.
- **الكائن** (object): نسخة حقيقية مصنوعة من القالب: «محرّك الكسّارة رقم 3».

لكل كائن **خصائص** (ما يملكه: الاسم، القدرة، الحالة) و**توابع** (ما يستطيع فعله: يشتغل، يتوقّف، يحسب استهلاكه).

> **معلومة**
>
> لقد استعملت الكائنات منذ البداية دون أن تدري! كل نص في بايثون كائن من الصنف `str`، ولهذا تكتب `"abc".upper()` : فـ `upper` تابع يملكه كل نص.

13.2 أوّل صنف لك

الفهرس

Python

```python
class Motor:
    def __init__(self, name, power_kw):
        self.name = name
        self.power_kw = power_kw
        self.running = False
        self.hours = 0

m1 = Motor("كسّارة", 250)
m2 = Motor("مروحة السحب", 75)
print(m1.name, m1.power_kw)
print(m2.name, m2.running)
```

الناتج

```
كسّارة 250
مروحة السحب False
```

ما هي `__init__` و `self`؟

- `__init__` دالة خاصة تسمّى **الباني**، تُنفَّذ تلقائيًّا عند إنشاء كل كائن جديد، ومهمّتها تجهيز خصائصه.

- `self` تعني «هذا الكائن نفسه». عندما نكتب `self.name = name` فنحن نقول: «اسم **هذا** المحرّك هو الاسم الذي أُعطي لي».

> **نصيحة ذهبية**
>
> اسم الصنف يبدأ بحرف كبير بالعادة: `Motor` و `Pump` و `Worker` ، بينما أسماء المتغيّرات والدوال بحروف صغيرة. هذا يساعدك على التمييز بينها من النظرة الأولى.

13.3 التوابع: ماذا يستطيع الكائن أن يفعل؟

الفهرس

التوابع دوال داخل الصنف، وأول معامل فيها دائمًا `self` :

Python

```python
class Motor:
    def __init__(self, name, power_kw):
        self.name = name
        self.power_kw = power_kw
        self.running = False
        self.hours = 0

    def start(self):
        self.running = True
        print(f"▶ {self.name} يعمل")

    def stop(self):
        self.running = False
        print(f"■ {self.name} متوقّف")

    def run(self, hours):
        if self.running:
            self.hours += hours

    def energy(self):
        return self.power_kw * self.hours

m = Motor("طاحونة", 900)
m.start()
m.run(6)
m.run(2)
m.stop()
m.run(5)          # لن تُحسب لأنه متوقّف
print(f"الطاقة: {m.energy():,} kWh")
```

الناتج

```
▶ طاحونة يعمل
■ طاحونة متوقّف
الطاقة: 7,200 kWh
```

لاحظ كيف أن الكائن **يتذكّر حالته**: يعرف إن كان يعمل، ويجمع ساعات تشغيله بنفسه.

13.4 الطباعة الجميلة بـ __str__

الفهرس

إذا طبعت كائنًا مباشرة فسترى شيئًا غريبًا مثل `<__main__.Motor object at 0x7f...>`. التابع الخاص `__str__` يحدّد كيف يُعرض الكائن:

Python

```python
class Pump:
    def __init__(self, code, flow):
        self.code = code
        self.flow = flow

    def __str__(self):
        return f"مضخّة {self.code} ({self.flow} m³/h)"

p = Pump("P-07", 120)
print(p)
```

الناتج

```
مضخّة P-07 (120 m³/h)
```

13.5 الوراثة

الفهرس

يمكن لصنف جديد أن **يرث** كل خصائص صنف موجود وتوابعه، ثم يضيف إليها أو يعدّلها. نكتب الصنف الأب بين قوسين:

Python

```python
class Machine:
    def __init__(self, name, power_kw):
        self.name = name
        self.power_kw = power_kw

    def describe(self):
        return f"{self.name}: {self.power_kw} kW"

class Conveyor(Machine):
    def __init__(self, name, power_kw, length_m):
        super().__init__(name, power_kw)
        self.length_m = length_m

    def describe(self):
        return super().describe() + f"، الطول {self.length_m} m"

items = [Machine("ضاغط هواء", 90), Conveyor("ناقل الكلنكر", 45, 120)]
for it in items:
    print(it.describe())
```

الناتج

```
ضاغط هواء: 90 kW
ناقل الكلنكر: 45 kW، الطول 120 m
```

- `Conveyor(Machine)` يعني: الناقل **هو نوع من** الآلات.
- `super().__init__(...)` تستدعي باني الأب ليجهّز الخصائص المشتركة.
- تعريف `describe` من جديد في الابن **يعدّل** سلوك الأب، وهذا يسمّى إعادة التعريف (override).



بايثو يقول:

هل يجب أن أستعمل الأصناف في كل برنامج؟ لا! للبرامج الصغيرة تكفي الدوال تمامًا. لكن عندما يكبر مشروعك وتكثر «الأشياء» فيه (آلات، عمّال، طلبيات...)، ستجد أن الأصناف ترتّب الفوضى بشكل سحري.

خلاصة الفصل

- `class Name:` لتعريف صنف، و `Name(...)` لإنشاء كائن منه.
- `__init__(self, ...)` تجهّز خصائص الكائن، و `self` تعني الكائن نفسه.
- التوابع دوال داخل الصنف، تُستدعى هكذا: `obj.method()` .
- `__str__` تحدّد شكل الكائن عند طباعته.
- `class Child(Parent):` للوراثة، و `super()` للوصول إلى الأب.

تحدّي الفصل

أنشئ صنفًا `BankAccount` فيه رصيد وتابعان `deposit` و `withdraw` . يجب أن يرفض السحب إذا كان المبلغ أكبر من الرصيد، وأن يحتفظ بسجلّ (قائمة) بكل العمليات. أضف تابعًا يطبع كشف الحساب.

الفصل

14

المشروع النهائي: مراقب الطاقة

برنامج حقيقي يحلّل استهلاك محرّكات مصنع ويقترح فرص التوفير

في هذا الفصل ستتعلّم

- تخطيط مشروع قبل كتابة أوّل سطر
- بناء صنف Motor يحسب استهلاكه
- طباعة تقرير منسّق وحفظه
- تنظيم البيانات في ملف CSV
- قراءة البيانات ومعالجة الأخطاء
- قائمة تفاعلية تجمع كل شيء

هذا هو الفصل الذي كنت تنتظره! كل ما تعلّمته سيجتمع هنا في برنامج واحد تفخر به.



14.1 فكرة المشروع

الفهرس

في أيّ مصنع، الكهرباء من أكبر بنود التكلفة. وقبل أن نوفّر، يجب أن نعرف **أين تذهب الطاقة**. سنكتب برنامجًا اسمه **مراقب الطاقة** يقوم بما يلي:

1. يقرأ قائمة المحرّكات وبياناتها من ملف CSV.
2. يحسب الاستهلاك اليومي لكل محرّك وتكلفته.
3. يرتّب المحرّكات من الأكثر استهلاكًا إلى الأقل، مع حصّة كل واحد.
4. يقترح فرص توفير بتركيب مغيّرات السرعة (VFD) على المراوح والمضخّات.
5. يحفظ التقرير في ملف يفتحه إكسل.

> **نصيحة ذهبية**
>
> المبرمجون المحترفون لا يبدؤون بكتابة الكود، بل بكتابة **قائمة المهام** كما فعلنا الآن. فإذا عرفت ماذا تريد بالضبط، أصبح الكود مجرّد ترجمة لأفكارك.

14.2 الخطوة 1: ملف البيانات

الفهرس

أنشئ بالمفكّرة (أو بإكسل مع الحفظ بصيغة CSV UTF-8) ملفًّا اسمه `motors.csv` بجانب برنامجك:

motors.csv

```
code,name,kind,power_kw,load,hours
M01,كسارة,crusher,250,0.80,16
M02,طاحونة الخام,mill,1800,0.90,22
M03,مروحة الفرن,fan,630,0.75,24
M04,مضخة الماء,pump,55,0.60,20
M05,ضاغط الهواء,compressor,160,0.70,18
M06,مروحة الطاحونة,fan,400,0.65,22
M07,ناقل الكلنكر,conveyor,45,0.85,24
```

العمود	معناه
`power_kw`	القدرة الاسمية للمحرّك بالكيلوواط
`load`	معامل الحمولة: نسبة القدرة المستعملة فعليًّا (0.80 = 80%)
`hours`	ساعات التشغيل في اليوم

14.3 الخطوة 2: الصنف Motor

الفهرس

كل محرّك كائن يعرف بياناته ويحسب استهلاكه بنفسه. الاستهلاك اليومي = القدرة × معامل الحمولة × الساعات:

energy_monitor.py — python

```python
import csv

PRICE = 4.2          # دينار لكل kWh
VFD_SAVING = 0.25    # توفير مغيّر السرعة على المراوح والمضخّات

class Motor:
    def __init__(self, code, name, kind, power_kw, load, hours):
        self.code = code
        self.name = name
        self.kind = kind
        self.power_kw = power_kw
        self.load = load
        self.hours = hours

    def daily_kwh(self):
        return self.power_kw * self.load * self.hours

    def daily_cost(self):
        return self.daily_kwh() * PRICE

    def vfd_candidate(self):
        return self.kind in ("fan", "pump")
```

معلومة

لماذا المراوح والمضخّات بالذات؟ لأن قدرتها تتناسب مع **مكعّب** السرعة (قوانين التشابه). فإذا خفّضت السرعة بـ 20% فقط، تنخفض القدرة بنحو 49%! ولهذا يكون مغيّر السرعة فيها أربح بكثير من الخنق بالصمّامات أو الريش. أما النسبة 25% في برنامجنا فتقدير متحفّظ، والقيمة الحقيقية تُقاس في الموقع.

14.4 الخطوة 3: قراءة الملف بأمان

الفهرس

دالة تقرأ كل سطر من الملف وتحوّله إلى كائن `Motor`، ولا تنهار إذا كان الملف غير موجود:

```python
def load_motors(path):
    motors = []
    try:
        with open(path, encoding="utf-8") as f:
            for r in csv.DictReader(f):
                motors.append(Motor(r["code"], r["name"], r["kind"],
                                    float(r["power_kw"]), float(r["load"]),
                                    float(r["hours"])))
    except FileNotFoundError:
        print(f"❌ الملف {path} غير موجود")
    return motors
```

14.5 الخطوة 4: التقرير وفرص التوفير

الفهرس

```python
def report(motors):
    total = sum(m.daily_kwh() for m in motors)
    print(f"{'الرمز':<7}{'المعدّة':<16}{'kWh/يوم':>10}{'الحصة':>8}")
    print("-" * 41)
    for m in sorted(motors, key=lambda m: m.daily_kwh(), reverse=True):
        share = m.daily_kwh() / total
        print(f"{m.code:<7}{m.name:<16}{m.daily_kwh():>10,.0f}{share:>8.1%}")
    print("-" * 41)
    print(f"المجموع: {total:,.0f} kWh/يوم = {total * PRICE:,.0f} دينار/يوم")
    return total

def savings(motors):
    print("\n💡 فرص التوفير بمغيّرات السرعة (VFD):")
    year = 0
    for m in motors:
        if m.vfd_candidate():
            kwh = m.daily_kwh() * VFD_SAVING * 330    # 330 يوم عمل
            year += kwh
            print(f"  {m.code} {m.name}: {kwh:,.0f} kWh/سنة")
    print(f"  التوفير السنوي ≈ {year * PRICE:,.0f} دينار")
```

لاحظ كم استعملنا مما تعلّمناه: سلاسل f مع التنسيق `:>10,.0f` و `:.1%`، والدالة `sorted` مع `lambda`، والمجموع التراكمي، والشرط...

14.6 الخطوة 5: الحفظ والقائمة الرئيسية

الفهرس

Python

```python
def save_report(motors, path="energy_report.csv"):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["code", "name", "kwh_day", "cost_day"])
        for m in motors:
            w.writerow([m.code, m.name, round(m.daily_kwh()),
round(m.daily_cost())])
    print(f"✅ حُفظ التقرير في {path}")

def main():
    motors = load_motors("motors.csv")
    if not motors:
        return
    while True:
        print("\n⚡ مراقب الطاقة  1) التقرير  2) التوفير  3) حفظ  4) خروج")
        choice = input("اختيارك: ")
        if choice == "1":
            report(motors)
        elif choice == "2":
            savings(motors)
        elif choice == "3":
            save_report(motors)
        elif choice == "4":
            print("إلى اللقاء 👋")
            break
        else:
            print("اختيار غير معروف")

if __name__ == "__main__":
    main()
```

معلومة

السطر `if __name__ == "__main__":` يعني: «شغّل `main()` فقط إذا شُغّل هذا الملف مباشرة». أما إذا استورده ملف آخر كوحدة (ليستعمل الصنف `Motor` مثلًا)، فلن تشتغل القائمة تلقائيًّا. عادة احترافية تجدها في كل المشاريع الكبيرة.

14.7 لحظة الحقيقة: التشغيل

الفهرس

احفظ كل الأجزاء في ملف واحد `energy_monitor.py` واضغط F5 :

الناتج

```
⚡ مراقب الطاقة ⚡  1) التقرير  2) التوفير  3) حفظ  4) خروج
اختيارك: 1
الرمز  المعدّة                kWh/يوم    الحصة
----------------------------------------
M02    طاحونة الخام           35,640    59.9%
M03    مروحة الفرن            11,340    19.1%
M06    مروحة الطاحونة          5,720     9.6%
M01    كسارة                   3,200     5.4%
M05    ضاغط الهواء             2,016     3.4%
M07    ناقل الكلنكر              918     1.5%
M04    مضخة الماء                660     1.1%
----------------------------------------
المجموع: 59,494 kWh/يوم = 249,875 دينار/يوم

⚡ مراقب الطاقة ⚡  1) التقرير  2) التوفير  3) حفظ  4) خروج
اختيارك: 2

💡 فرص التوفير بمغيّرات السرعة (VFD):
  M03 مروحة الفرن: 935,550 kWh/سنة
  M04 مضخة الماء: 54,450 kWh/سنة
  M06 مروحة الطاحونة: 471,900 kWh/سنة
  التوفير السنوي ≈ 6,139,980 دينار
```

في أقل من 90 سطرًا، صنعت أداة تكشف أن **طاحونة الخام وحدها تستهلك 60% من الطاقة**، وأن مغيّرات السرعة على ثلاث معدّات فقط قد توفّر ملايين الدنانير سنويًّا. هذه هي قوّة البرمجة!

من أرض المصنع

لتطوير المشروع أكثر، يمكنك:

- قراءة القراءات الحقيقية من عدّادات الطاقة أو من نظام الإشراف (SCADA) بدل القيم الثابتة.
- استعمال `pandas` لتحليل سجلّ عام كامل، و `matplotlib` لرسم مخطّط دائري لحصص الاستهلاك.
- إضافة تعريفة الساعات (ذروة / خارج الذروة) لحساب التكلفة بدقّة.
- إرسال تقرير يومي آليًّا بالبريد الإلكتروني إلى مسؤول الطاقة.



بايثو يقول:

لقد وصلت إلى نهاية الرحلة الأساسية... وأنا فخور بك جدًّا! من `print("مرحبًا")` إلى برنامج يحلّل مصنعًا كاملًا. الآن أنت مبرمج بايثون حقيقي، ولا تدع أحدًا يقول لك العكس 🐍💛

تحدّي الفصل

أضف إلى المشروع خيارًا خامسًا في القائمة: «إضافة محرّك جديد»، يسأل المستخدم عن بيانات المحرّك (مع حماية الإدخال من الأخطاء)، ثم يضيفه إلى القائمة **وإلى الملف** `motors.csv` حتى لا يضيع عند إغلاق البرنامج.

الورقة المرجعية السريعة

اطبع هذه الصفحات وعلّقها بجانب حاسوبك، ففيها خلاصة الكتاب كله في لمحة.

1 الأساسيات

الفهرس

الموضوع	الصيغة
طباعة	`print("نص", x, sep=" ", end="\n")`
إدخال	`x = input("سؤال ")` (نص دائمًا)
تعليق	`# ملاحظة`
متغيّر	`name = "قيمة"`
الأنواع	`int` و `float` و `str` و `bool`
التحويل	`int(x)` و `float(x)` و `str(x)`
النوع	`type(x)`
سلسلة f	`f"القيمة {x:.2f}"`

2 العمليات

الفهرس

النوع	الرموز
حسابية	`+ - * / // % **`
اختصارات	`+= -= *= /=`
مقارنة	`= ≠ > < ≥ ≤`
منطقية	`and` و `or` و `not`
انتماء	`in` و `not in`

3 الشروط والحلقات

الفهرس

Python

```python
if x > 10:
    ...
elif x > 5:
    ...
else:
    ...

for i in range(1, 11):      # من 1 إلى 10
    if i == 3:
        continue            # الدورة التالية
    if i == 8:
        break               # اخرج من الحلقة

while condition:
    ...
```

4 النصوص

الفهرس

العملية	المثال
الطول	`len(s)`
فهرس / قصّ	`s[0]` و `s[-1]` و `s[2:5]` و `s[::-1]`
تحويل الحالة	`s.upper()` و `s.lower()`
تنظيف	`s.strip()`
استبدال	`s.replace("a", "b")`
تقسيم / دمج	`s.split(",")` و `"-".join(lst)`
بحث	`"x" in s` و `s.find("x")` و `s.count("x")`

5 القوائم والقواميس والمجموعات

الفهرس

العملية	القائمة `[ ]`	القاموس `{k: v}`
إنشاء	`a = [1, 2, 3]`	`d = {"a": 1}`
قراءة	`a[0]`	`d["a"]` أو `d.get("a", 0)`
إضافة	`a.append(4)`	`d["b"] = 2`
حذف	`a.remove(4)` و `a.pop()`	`del d["b"]`
مرور	`for x in a:`	`for k, v in d.items():`
وجود	`3 in a`	`"a" in d`

- إحصاء: `sum(a)` و `max(a)` و `min(a)` و `sorted(a)` و `a.sort(reverse=True)`
- فهم القوائم: `[x * 2 for x in a if x > 1]`
- مجموعة بلا تكرار: `set(a)` ، واتحاد `|` ، وتقاطع `&` ، وفرق `-`

6 الدوال والأصناف

الفهرس

Python `python`

```python
def area(w, h=1):
    """وصف الدالة."""
    return w * h

class Motor:
    def __init__(self, name, kw):
        self.name = name
        self.kw = kw

    def energy(self, hours):
        return self.kw * hours

    def __str__(self):
        return f"{self.name} ({self.kw} kW)"
```

7 الملفات والأخطاء

الفهرس

Python

```python
with open("data.txt", "w", encoding="utf-8") as f:   # r / w / a
    f.write("سطر\n")

try:
    x = float(input("رقم: "))
except ValueError:
    print("ليس رقمًا")
finally:
    print("انتهى")
```

8 نافذة الأوامر

الفهرس

الأمر	فائدته
`python --version`	معرفة الإصدار
`python file.py`	تشغيل ملف
`pip install name`	تثبيت مكتبة
`pip list`	المكتبات المثبّتة
`Ctrl` `C`	إيقاف برنامج عالق

حلول التحدّيات

قبل أن تنظر إلى الحل، أعطِ نفسك فرصة حقيقية: حاول 20 دقيقة على الأقل. وتذكّر أن حلّك قد يكون مختلفًا عن حلّي وصحيحًا تمامًا، ففي البرمجة طرق كثيرة توصل إلى النتيجة نفسها.

1 تحدّي الفصل 1: بطاقة التعريف

الفهرس

card.py — python

```python
print("الاسم: سفيان مرابطي")
print("المهنة: مهندس طاقة")
print("المدينة: الجزائر")
print("الهواية: البرمجة")
print("**********")
```

2 تحدّي الفصل 2: متغيّرات المحرّك

الفهرس

Python — python

```python
name = "مضخة التبريد"
power = 37.5
poles = 4
running = True
print(name, type(name))
print(power, type(power))
print(poles, type(poles))
print(running, type(running))
```

الناتج

```
مضخة التبريد <class 'str'>
37.5 <class 'float'>
4 <class 'int'>
True <class 'bool'>
```

3 تحدّي الفصل 3: استهلاك المصباح

الفهرس

Python

```python
watts = 60
hours = 5
days = 30
kwh = watts * hours * days / 1000
cost = kwh * 4.2
print("الاستهلاك:", round(kwh, 2), "kWh")
print("التكلفة:", round(cost, 2), "دينار")
```

الناتج

```
الاستهلاك: 9.0 kWh
التكلفة: 37.8 دينار
```

4 تحدّي الفصل 4: رمز القطعة

الفهرس

Python

```python
code = "pump-0457-dz"
kind, number, country = code.split("-")
kind = kind.upper()
number = int(number)
country = country.upper()
print(kind, number, country)
print(f"القطعة {kind} رقم {number} من {country}")
```

الناتج

```
PUMP 457 DZ
القطعة PUMP رقم 457 من DZ
```

5 تحدّي الفصل 5: فاتورة البلاط

الفهرس

Python

```python
length = float(input("الطول (م): "))
width = float(input("العرض (م): "))
price = float(input("سعر المتر المربّع: "))

area = length * width
total = area * price
print("-" * 26)
print(f"{'المساحة':<14}{area:>9.2f} م²")
print(f"{'السعر':<14}{total:>9.2f} دج")
print(f"{'مع الاحتياط':<14}{total * 1.10:>9.2f} دج")
```

6 تحدّي الفصل 6: فاتورة الشرائح

الفهرس

Python

```python
kwh = float(input("الاستهلاك (kWh): "))

if kwh ⩽ 125:
    bill = kwh * 1.8
elif kwh ⩽ 250:
    bill = 125 * 1.8 + (kwh - 125) * 4.2
else:
    bill = 125 * 1.8 + 125 * 4.2 + (kwh - 250) * 5.5

print(f"المبلغ: {bill:.2f} دينار")
```

الناتج

```
الاستهلاك (kWh): 300
المبلغ: 1025.00 دينار
```

7 تحدّي الفصل 7: قراءات الحرارة

الفهرس

Python

```python
count = 0
total = 0
highest = None

while True:
    t = float(input("القراءة: (0 للإنهاء) "))
    if t == 0:
        break
    count += 1
    total += t
    if highest is None or t > highest:
        highest = t

if count > 0:
    print(f"العدد: {count}، المتوسّط: {total / count:.1f}، الأعلى: {highest}")
else:
    print("لم تُدخل أيّ قراءة")
```

8 تحدّي الفصل 8: ساعات العمّال

الفهرس

Python

```python
workers = ["علي", "سارة", "يوسف", "نادية"]
hours = [38, 45, 41, 36]

for w, h in zip(workers, hours):
    print(f"{w}: {h} ساعة")

overtime = [w for w, h in zip(workers, hours) if h > 40]
print("تجاوزوا 40 ساعة:", overtime)

best = workers[hours.index(max(hours))]
print("الأكثر عملًا:", best)
```

الناتج

```
علي: 38 ساعة
سارة: 45 ساعة
يوسف: 41 ساعة
نادية: 36 ساعة
تجاوزوا 40 ساعة: ['سارة', 'يوسف']
الأكثر عملًا: سارة
```

9 تحدّي الفصل 9: دليل الهاتف

الفهرس

Python

```python
phones = {"رئيس الورشة": "0550 12 34 56", "الكهربائي": "0661 98 76 54"}

while True:
    print("\n1) خروج  4) عرض الكل  3) إضافة  2) بحث")
    choice = input("اختيارك: ")
    if choice == "1":
        name = input("الاسم: ")
        print(phones.get(name, "غير موجود"))
    elif choice == "2":
        name = input("الاسم: ")
        phones[name] = input("الرقم: ")
        print("✅ أُضيف")
    elif choice == "3":
        for name, num in phones.items():
            print(f"{name}: {num}")
    elif choice == "4":
        break
```

10 تحدّي الفصل 10: الأعداد الأوّلية

الفهرس

Python

```python
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True

print([n for n in range(1, 51) if is_prime(n)])
```

الناتج

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

يكفي أن نبحث عن القواسم حتى الجذر التربيعي للعدد، لأن كل قاسم أكبر منه يقابله قاسم أصغر منه.

11 تحدّي الفصل 11: كلمة السرّ

الفهرس

Python

```python
import random
import string

chars = string.ascii_letters + string.digits + string.punctuation
password = [random.choice(string.ascii_uppercase),
            random.choice(string.ascii_lowercase),
            random.choice(string.digits),
            random.choice(string.punctuation)]
password += [random.choice(chars) for _ in range(8)]
random.shuffle(password)
print("".join(password))
```

الناتج

```
k7#Qm!2xR_a9
```

اخترنا حرفًا واحدًا على الأقل من كل نوع ثم خلطنا الحروف، لنضمن أن كلمة السرّ تحتوي على كل الأنواع.

12 تحدّي الفصل 12: تقرير الطاقة

الفهرس

Python

```python
import csv

try:
    total = 0
    with open("energy.csv", encoding="utf-8") as f:
        for r in csv.DictReader(f):
            total += float(r["kw"]) * float(r["hours"])
    with open("report.txt", "w", encoding="utf-8") as out:
        out.write(f"الطاقة الإجمالية: {total:,.0f} kWh\n")
    print("✅ كُتب التقرير")
except FileNotFoundError:
    print("❌ لم أجد الملف energy.csv")
```

13 تحدّي الفصل 13: الحساب البنكي

الفهرس

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = []

    def deposit(self, amount):
        self.balance += amount
        self.history.append(f"+{amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("❌ الرصيد غير كافٍ")
            return
        self.balance -= amount
        self.history.append(f"-{amount}")

    def statement(self):
        print(f"حساب {self.owner}: {' '.join(self.history)} → الرصيد {self.balance}")

acc = BankAccount("مريم")
acc.deposit(5000)
acc.withdraw(1200)
acc.withdraw(9000)
acc.statement()
```

الناتج

```
❌ الرصيد غير كافٍ
حساب مريم: +5000 -1200 → الرصيد 3800
```

14 تحدّي الفصل 14: إضافة محرّك

الفهرس

نضيف دالة جديدة تستعمل `ask_number` من الفصل الثاني عشر، ثم الخيار `"5"` في القائمة:

Python

```python
def ask_number(question):
    while True:
        try:
            return float(input(question))
        except ValueError:
            print("⚠ هذا ليس رقمًا")

def add_motor(motors, path="motors.csv"):
    m = Motor(input("الرمز: "), input("الاسم: "), input("النوع (fan/pump/...): "),
              ask_number("القدرة kW: "), ask_number("معامل الحمولة: "),
              ask_number("الساعات/يوم: "))
    motors.append(m)
    with open(path, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([m.code, m.name, m.kind, m.power_kw, m.load,
m.hours])
    print(f"✅ أُضيف {m.name}")

# وفي main:
#     elif choice == "5":
#         add_motor(motors)
```



ماذا بعد؟

مبروك! إذا وصلت إلى هنا وجرّبت الأمثلة بيدك، فأنت تعرف الآن أساسيات بايثون التي يقوم عليها كل شيء آخر: من مواقع الإنترنت إلى الذكاء الاصطناعي. لكن الرحلة لم تنتهِ، بل بدأت للتوّ.

1 اختر طريقك

الفهرس

المجال	ابدأ بـ	ماذا ستصنع
تحليل البيانات	`pandas` و `matplotlib`	تقارير ومنحنيات من ملفات إكسل الضخمة
الذكاء الاصطناعي	`scikit-learn` ثم `PyTorch`	نماذج تتنبّأ بالأعطال والاستهلاك
الأتمتة	`openpyxl` و `os` و `smtplib`	برامج تنجز الأعمال المكتبية المتكرّرة
مواقع الإنترنت	`Flask` أو `Django`	تطبيقات ويب كاملة
الصناعة والأجهزة	`pyserial` و `pymodbus`	التواصل مع الحسّاسات والمتحكّمات
العلوم والهندسة	`numpy` و `scipy`	محاكاة وحلّ معادلات وتحليل إشارات

2 نصائح للاستمرار

الفهرس

- **برمج كل يوم** ولو 20 دقيقة. الاستمرار أهم من المدّة.
- **اصنع مشاريع صغيرة تخصّك**: حاسبة لعملك، منظّم لملفاتك، لعبة لأبنائك...
- **اقرأ أكواد الآخرين** على GitHub، فهي مدرسة مجانية.
- **لا تخجل من البحث**: كل المبرمجين المحترفين يبحثون في الإنترنت يوميًّا، والرسائل الحمراء تُنسخ وتُلصق في محرّك البحث!
- **علّم غيرك**: أفضل طريقة لتثبيت ما تعلّمته أن تشرحه لشخص آخر.



بايثو يقول:

كان شرفًا لي أن أرافقك من أوّل `print` إلى آخر صنف. سأشتاق إليك... لكنني سأكون دائمًا في السطر الأول من كل برنامج تكتبه. إلى اللقاء يا صديقي المبرمج! 🐍

3 كلمة أخيرة

الفهرس

كتبت هذا الكتاب لأني أؤمن أن البرمجة لم تعد ترفًا للمتخصّصين، بل مهارة أساسية لكل مهندس وتقني وطالب وأستاذ، مثل القراءة والكتابة تمامًا. وأتمنّى أن يكون قد فتح لك بابًا جديدًا.

إذا أعجبك الكتاب، فشاركه مع من يحتاجه، ولا تتردّد في مراسلتي بأسئلتك واقتراحاتك وصور مشاريعك، فلا شيء يسعدني أكثر من رؤية ما صنعته بيديك.

د. سفيان مرابطي



لم تكتب سطرًا برمجيًّا في حياتك؟ ممتاز!

هذا الكتاب كُتب لك أنت. سنبدأ معًا من تثبيت بايثون على جهازك، ثم نتعلّم المتغيرات والشروط والحلقات والقوائم والدوال والملفات والبرمجة الكائنية، خطوة خطوة، بأمثلة ملوّنة وتمارين وتحدّيات، حتى تبني بيديك مشروعًا حقيقيًّا من عالم الصناعة.

- شرح بسيط بلا تعقيد
- أكثر من 100 مثال جاهز للتجربة
- تمارين وحلول وتحدّيات
- ورقة مرجعية سريعة

تأليف د. سفيان مرابطي

Dr Merabti